DevOps Masterclass for Beginners

Chapter 1: Introduction to DevOps

Learning Objective: Introduce DevOps concepts, culture, and lifecycle
Skills Covered: Understanding DevOps philosophy, Agile/Scrum basics, SDLC, DevOps roles.
Prerequisites: Basic understanding of software development and IT operations.
Estimated Time: 3–4 hours
Number of Lessons: 5

1. **Lesson 1.1: What is DevOps?** (Beginner)
2. **Learning Outcome:** Define DevOps as the integration of development and operations for faster, reliable delivery.
3. **Topics Covered:** DevOps definition, goals (speed, reliability), cultural philosophy.
4. **Hands-on Lab:** Set up a simple Git repository and commit a code change.
5. **Mini Assignment:** Research and list 3 benefits of DevOps from case studies.
6. **Common Mistakes:** Thinking DevOps is just tools; ignoring culture.
7. **Interview Questions:** *What is DevOps and why is it important?*
8. **Quiz Questions:** *MCQ on DevOps definition and benefits*.
9. **Lesson 1.2: Why DevOps?** (Beginner)
10. **Learning Outcome:** Explain the business and technical drivers for DevOps adoption.
11. **Topics Covered:** Challenges in traditional SDLC, need for faster releases, improved quality【5†L57-L66】.
12. **Hands-on Lab:** Compare manual vs automated deployment of a simple app.
13. **Mini Assignment:** Find 2 examples of companies that benefited from DevOps.
14. **Common Mistakes:** Overlooking cultural change or assuming DevOps solves all problems.
15. **Interview Questions:** *What problems does DevOps solve?*
16. **Quiz Questions:** *True/False on DevOps benefits (e.g. reliability, scalability)*.
17. **Lesson 1.3: Software Development Life Cycle (SDLC)** (Beginner)
18. **Learning Outcome:** Describe SDLC phases and their purpose.
19. **Topics Covered:** Planning, design, implementation, testing, deployment, maintenance.
20. **Hands-on Lab:** Draw an SDLC workflow diagram and map DevOps integrations.

21. **Mini Assignment:** Compare traditional SDLC vs Agile vs DevOps approaches.
22. **Common Mistakes:** Skipping testing or maintenance; not revisiting phases.
23. **Interview Questions:** *Explain the main phases of SDLC*.
24. **Quiz Questions:** *Match SDLC phase to description*.

25. **Lesson 1.4: Agile and Scrum Basics** (Beginner)

26. **Learning Outcome:** Explain Agile values and Scrum framework basics.

27. **Topics Covered:** Agile manifesto, Scrum roles (PO, Scrum Master, Team) and ceremonies.
28. **Hands-on Lab:** Create a simple product backlog and conduct a mock sprint planning.
29. **Mini Assignment:** Practice writing user stories and acceptance criteria.
30. **Common Mistakes:** Confusing Scrum with all of Agile; ignoring retrospectives.
31. **Interview Questions:** *What is a Scrum sprint and who are the key roles?*
32. **Quiz Questions:** *Identify correct Scrum events or roles (MCQ).*

33. **Lesson 1.5: DevOps Culture and Roles** (Beginner)

34. **Learning Outcome:** Understand DevOps culture (collaboration, automation) and common roles.

35. **Topics Covered:** Collaboration between dev & ops, CAMS (culture, automation, measurement, sharing), DevOps engineer role.
36. **Hands-on Lab:** Design a RACI matrix for a DevOps project team.
37. **Mini Assignment:** Interview a colleague about current team silos and how to bridge them.
38. **Common Mistakes:** Thinking only tools matter; neglecting communication or shared responsibility.
39. **Interview Questions:** *How do developers and operations collaborate in DevOps?*
40. **Quiz Questions:** *Fill-in-the-blank on CAMS or roles of a DevOps engineer*.

Chapter 2: Operating Systems & Linux

Learning Objective: Master Linux fundamentals and system administration basics.
Skills Covered: Linux installation, file system, shell usage, scripting, user management, processes, services, cron, networking commands.
Prerequisites: None (absolute beginner).
Estimated Time: 8–10 hours
Number of Lessons: 10

1. **Lesson 2.1: Linux Installation & Distros** (Beginner)

2. **Learning Outcome:** Install and boot a Linux distribution; understand common distros (Ubuntu, CentOS).
3. **Topics Covered:** Downloading ISO, installation steps, virtualization vs bare-metal.
4. **Hands-on Lab:** Install Ubuntu or CentOS in a VM.
5. **Mini Assignment:** List differences between at least two Linux distros.
6. **Common Mistakes:** Ignoring disk partition options.
7. **Interview Questions:** *How do you install a Linux server?*
8. **Quiz Questions:** *Multiple choice on Linux distro features.*
9. **Lesson 2.2: Linux File System and Hierarchy** (Beginner)
10. **Learning Outcome:** Navigate and describe the Linux FHS (Filesystem Hierarchy Standard).
11. **Topics Covered:** Root (/), /home, /etc, /var, /usr, /bin, /boot, mounting volumes.
12. **Hands-on Lab:** List and explain contents of key directories on a Linux VM.
13. **Mini Assignment:** Create and mount a new directory or disk.
14. **Common Mistakes:** Storing data in /root or /tmp improperly.
15. **Interview Questions:** *What is the purpose of /etc or /var?*
16. **Quiz Questions:** *Match directory to description (e.g. /var/log holds logs).*
17. **Lesson 2.3: Linux Shell Basics (Bash)** (Beginner)
18. **Learning Outcome:** Use basic shell commands and navigate the CLI (Command Line Interface).
19. **Topics Covered:** Shell prompt, `ls`, `cd`, `cp`, `mv`, `rm`, `cat`, `less`, pipes (|), redirection (>, >>).
20. **Hands-on Lab:** Practice file and directory commands; combine with pipes and redirects.
21. **Mini Assignment:** Write commands to list files by date, filter logs with `grep`.
22. **Common Mistakes:** Deleting files without backups; confusing > vs >>.
23. **Interview Questions:** *How do you find a string in files on Linux?*
24. **Quiz Questions:** *What does `ls -l` show?*
25. **Lesson 2.4: Bash Scripting** (Intermediate)
26. **Learning Outcome:** Write basic Bash scripts to automate tasks.
27. **Topics Covered:** Shebang (#!), variables, conditionals, loops, permissions (chmod +x), script debugging.
28. **Hands-on Lab:** Write a script that backs up a directory to a timestamped tarball.
29. **Mini Assignment:** Write a script to monitor disk usage and email warning if threshold exceeded.

30. **Common Mistakes:** Forgetting execute permission or proper quoting.
31. **Interview Questions:** *How do you pass arguments to a shell script?*
32. **Quiz Questions:** *Identify errors in a simple script snippet.*

33. **Lesson 2.5: Users and Groups** (Beginner)

34. **Learning Outcome:** Manage Linux users and groups for access control.

35. **Topics Covered:** `useradd`, `groupadd`, `/etc/passwd`, `/etc/group`, switching users (`su`), `sudo`.
36. **Hands-on Lab:** Create a new user, assign to a group, and set a password.
37. **Mini Assignment:** Configure `sudo` for a non-root user.
38. **Common Mistakes:** Leaving root account for routine tasks; wrong file permissions on /etc files.
39. **Interview Questions:** *How do you add a user to a group?*
40. **Quiz Questions:** *Multiple choice on user management commands.*

41. **Lesson 2.6: File Permissions and Ownership** (Beginner)

42. **Learning Outcome:** Explain and use Linux permission model (owner, group, others).

43. **Topics Covered:** Read/write/execute bits (`rwx`), `chmod`, `chown`, `chgrp`, numeric vs symbolic modes.
44. **Hands-on Lab:** Change permissions to allow group write on a project directory.
45. **Mini Assignment:** Practice `chmod` and predict permission string outcomes.
46. **Common Mistakes:** Overly permissive (`chmod 777`) on sensitive files.
47. **Interview Questions:** *What does the execute bit mean on a directory?*
48. **Quiz Questions:** *Calculate `chmod` value for given perms.*

49. **Lesson 2.7: Processes and Job Management** (Beginner)

50. **Learning Outcome:** View and control Linux processes.

51. **Topics Covered:** `ps`, `top`, `htop`, `kill`, foreground (`fg`), background (`&`), `nice` priorities.
52. **Hands-on Lab:** Start a background job and bring it to foreground; kill a process.
53. **Mini Assignment:** Write a command sequence to find and kill a process by name.
54. **Common Mistakes:** Killing wrong process ID; forgetting & runs jobs in background.
55. **Interview Questions:** *How do you list running processes?*
56. **Quiz Questions:** *True/False on process management commands.*

57. **Lesson 2.8: Services and Systemd** (Intermediate)

58. **Learning Outcome:** Manage Linux services using Systemd.

59. **Topics Covered:** `systemctl` (start/stop/status/enable), service units, runlevels/targets, `service` command on SysV.
60. **Hands-on Lab:** Enable and start the `ssh` service at boot; check its status.
61. **Mini Assignment:** Create a simple custom service file to run a script at startup.
62. **Common Mistakes:** Not reloading daemon after new service files; mixing old `init` scripts.
63. **Interview Questions:** *What is systemd and how do you use it?*
64. **Quiz Questions:** *Multiple choice on systemctl options.*

65. **Lesson 2.9: Cron Jobs and Scheduling** (Beginner)

66. **Learning Outcome:** Schedule repetitive tasks using cron.

67. **Topics Covered:** `crontab`, cron schedule syntax (minute, hour, day, etc), `/etc/cron.*`, `anacron`.
68. **Hands-on Lab:** Set up a cron job to rotate logs daily or run backups weekly.
69. **Mini Assignment:** Write cron entries for various schedules.
70. **Common Mistakes:** Wrong cron syntax or forgetting to set PATH in cron.
71. **Interview Questions:** *How do you check a user's cron jobs?*
72. **Quiz Questions:** *Build a cron schedule for "every Monday at 3am".*

73. **Lesson 2.10: Basic Networking Commands (Linux)** (Beginner)

    - **Learning Outcome:** Use Linux networking tools to diagnose connectivity.
    - **Topics Covered:** `ifconfig/ip addr`, `ping`, `netstat/ss`, `traceroute`, `nslookup/dig`, `ssh`.
    - **Hands-on Lab:** Use `ping` and `traceroute` to test network paths; fetch DNS info with `dig`.
    - **Mini Assignment:** Collect network configuration details of your VM.
    - **Common Mistakes:** Not running as root for certain network commands; ignoring IPv6 addresses.
    - **Interview Questions:** *How do you find your machine's IP address?*
    - **Quiz Questions:** *Identify output of `ping` or `netstat -tulpn`.*

Chapter 3: Networking Fundamentals

Learning Objective: Cover core networking concepts for DevOps (IP, DNS, HTTP, security).
Skills Covered: TCP/IP, DNS, HTTP/HTTPS, SSH, FTP/SFTP, load balancing, reverse proxy, firewalls.
Prerequisites: Basic networking knowledge (high school level).
Estimated Time: 6–8 hours
Number of Lessons: 9

1. **Lesson 3.1: IP Addressing and TCP/IP** (Beginner)

2. **Learning Outcome:** Understand IP addresses and the TCP/IP model.
3. **Topics Covered:** IPv4 vs IPv6, subnet masks, TCP vs UDP【53†L304-L312】, OSI vs TCP/IP layers.
4. **Hands-on Lab:** Use `ip addr` to view network config; configure a static IP on a VM.
5. **Mini Assignment:** Calculate network address given IP and mask.
6. **Common Mistakes:** Confusing network and host portions, mixing up bits and bytes.
7. **Interview Questions:** *What is the difference between TCP and UDP?*
8. **Quiz Questions:** *Multiple choice on IP classes or protocol differences.*
9. **Lesson 3.2: DNS (Domain Name System)** (Beginner)
10. **Learning Outcome:** Explain how DNS translates hostnames to IPs.
11. **Topics Covered:** DNS hierarchy (root, TLD, authoritative), record types (A, CNAME, MX).
12. **Hands-on Lab:** Use `dig` or `nslookup` to query domain records and interpret output.
13. **Mini Assignment:** Map a given hostname to its IP using DNS tools.
14. **Common Mistakes:** Forgetting to flush DNS cache; not understanding TTL.
15. **Interview Questions:** *How does DNS resolution work?*
16. **Quiz Questions:** *True/False: CNAME records point to other domain names.*
17. **Lesson 3.3: HTTP and HTTPS** (Beginner)
18. **Learning Outcome:** Understand web protocols HTTP and secure HTTPS.
19. **Topics Covered:** HTTP request/response, status codes, TLS/SSL basics, ports (80, 443).
20. **Hands-on Lab:** Use `curl` to fetch a web page via HTTP and HTTPS; inspect TLS certificate with OpenSSL.
21. **Mini Assignment:** Visit a site via both protocols; note differences.
22. **Common Mistakes:** Ignoring certificate warnings; mixing up port numbers.
23. **Interview Questions:** *What is the purpose of TLS in HTTPS?*
24. **Quiz Questions:** *Select correct HTTP status code for "Not Found".*
25. **Lesson 3.4: SSH (Secure Shell)** (Beginner)
26. **Learning Outcome:** Use SSH for secure remote access【49†L1105-L1108】.
27. **Topics Covered:** Key-based authentication, `ssh`, `scp`, basic SSH config.
28. **Hands-on Lab:** Generate SSH key pair; log into a server with keys.
29. **Mini Assignment:** Set up passwordless SSH login from one VM to another.
30. **Common Mistakes:** Open SSH to world with weak keys; not securing private key.

31. **Interview Questions:** *How does SSH authentication work?*
32. **Quiz Questions:** *True/False on SSH port number and cipher usage.*

33. **Lesson 3.5: FTP and SFTP** (Beginner)

34. **Learning Outcome:** Understand file transfer protocols FTP and SFTP.

35. **Topics Covered:** Differences FTP (unencrypted) vs SFTP/FTPS (secure), anonymous FTP.
36. **Hands-on Lab:** Use `sftp` to transfer a file to a remote server.
37. **Mini Assignment:** Explain why SFTP is preferred over FTP.
38. **Common Mistakes:** Running plain FTP over the internet; forgetting passive mode.
39. **Interview Questions:** *Why is SFTP more secure than FTP?*
40. **Quiz Questions:** *Fill-in: Default port for SFTP (22).*

41. **Lesson 3.6: Load Balancers** (Beginner)

42. **Learning Outcome:** Explain how load balancers distribute traffic【55†L51-L59】.

43. **Topics Covered:** Round-robin, least connections, sticky sessions, hardware vs software LB.
44. **Hands-on Lab:** Configure Nginx (or HAProxy) as a simple HTTP load balancer.
45. **Mini Assignment:** Design an LB setup for a three-server web cluster.
46. **Common Mistakes:** Single point of failure if LB not redundant.
47. **Interview Questions:** *How does a load balancer improve availability?*
48. **Quiz Questions:** *True/False on load balancer session persistence.*

49. **Lesson 3.7: Reverse Proxy** (Beginner)

50. **Learning Outcome:** Understand reverse proxies and their uses【58†L72-L75】.

51. **Topics Covered:** Nginx/Apache as reverse proxy, caching, SSL termination.
52. **Hands-on Lab:** Set up Nginx to reverse-proxy traffic to an internal app.
53. **Mini Assignment:** List benefits of using a reverse proxy in front of web servers.
54. **Common Mistakes:** Leaving open proxies that allow misuse.
55. **Interview Questions:** *What is a reverse proxy and why use it?*
56. **Quiz Questions:** *MCQ on reverse vs forward proxy.*

57. **Lesson 3.8: Firewalls and Network Security** (Intermediate)

58. **Learning Outcome:** Describe basic network security concepts (firewalls, NAT).

59. **Topics Covered:** iptables/ufw rules, security groups (cloud firewalls), port filtering.
60. **Hands-on Lab:** Configure a simple iptables rule to block a port.
61. **Mini Assignment:** Draft firewall rules for a web server.

62. **Common Mistakes:** Open too many ports; forgetting to restrict SSH.
63. **Interview Questions:** *How do you secure a Linux server's network traffic?*
64. **Quiz Questions:** *True/False on default firewall policies (allow/deny).*

65. **Lesson 3.9: Network Troubleshooting Tools** (Intermediate)

66. **Learning Outcome:** Use tools to diagnose network issues.

67. **Topics Covered:** `ping`, `traceroute`, `netstat`, `tcpdump`, `nslookup`.
68. **Hands-on Lab:** Simulate a network outage and use `ping`/`traceroute` to find the break.
69. **Mini Assignment:** Capture packets on interface to identify HTTP traffic.
70. **Common Mistakes:** Not checking host vs network connectivity first.
71. **Interview Questions:** *Which tool would you use to capture packets?*
72. **Quiz Questions:** *Scenario-based MCQ on network issue identification.*

Chapter 4: Git & GitHub

Learning Objective: Teach version control with Git and collaborative workflows on GitHub.
Skills Covered: Git basics (init, clone, commit), branching, merging, rebasing, pull requests, GitHub workflows, best practices.
Prerequisites: Linux shell basics, text editing.
Estimated Time: 5–6 hours
Number of Lessons: 7

1. **Lesson 4.1: Git Basics** (Beginner)

2. **Learning Outcome:** Initialize and manage a Git repository; perform commits【63†L1171-L1180】.

3. **Topics Covered:** `git init`, `clone`, `add`, `commit`, `status`, `log`【63†L1171-L1180】.
4. **Hands-on Lab:** Create a Git repo for a sample project and make several commits.
5. **Mini Assignment:** Revert to a previous commit using `git checkout` or `git revert`.
6. **Common Mistakes:** Forgetting to commit often; committing large binary files.
7. **Interview Questions:** *How do you undo the last commit?*
8. **Quiz Questions:** *MCQ on Git commands and their functions.*

9. **Lesson 4.2: Branching in Git** (Intermediate)

10. **Learning Outcome:** Use Git branches to develop features independently.

11. **Topics Covered:** `git branch`, `checkout`, feature branching, Git flow concepts, remote tracking.
12. **Hands-on Lab:** Create feature branches, switch between them, and merge changes.

13. **Mini Assignment:** Implement a small feature in a branch and merge into `main`.
14. **Common Mistakes:** Committing to `main` by accident; long-lived branches diverging.
15. **Interview Questions:** *Why use branches in Git?*
16. **Quiz Questions:** *Identify commands to list branches*.

17. **Lesson 4.3: Merging and Rebasing** (Intermediate)

18. **Learning Outcome:** Integrate changes using merge and rebase【66†L91-L99】.

19. **Topics Covered:** `git merge` (fast-forward vs 3-way), resolving merge conflicts, `git rebase` for linear history【66†L91-L99】.
20. **Hands-on Lab:** Create a merge conflict scenario and resolve it; rebase a feature branch onto updated `main`.
21. **Mini Assignment:** Practice `git rebase` with interactive to squash commits.
22. **Common Mistakes:** Rebasing public/shared branches; losing work when rebasing incorrectly.
23. **Interview Questions:** *Explain difference between merge and rebase*.
24. **Quiz Questions:** *True/False on rebase rewriting history*.

25. **Lesson 4.4: Pull Requests and Code Review** (Beginner)

26. **Learning Outcome:** Collaborate via pull requests (PRs) on GitHub for code review【68†L1178-L1186】.

27. **Topics Covered:** Creating PRs, branch protection, review comments, merging PRs【68†L1178-L1186】【68†L1184-L1192】.
28. **Hands-on Lab:** Open a PR on GitHub for a forked repo; simulate reviews.
29. **Mini Assignment:** Respond to a review comment and update a PR.
30. **Common Mistakes:** Merging without review; not linking issues to PRs.
31. **Interview Questions:** *What is a pull request?*
32. **Quiz Questions:** *Multiple choice on GitHub PR workflow*.

33. **Lesson 4.5: GitHub Workflow** (Intermediate)

34. **Learning Outcome:** Follow a collaborative GitHub development workflow.

35. **Topics Covered:** Forking vs branching, feature branches, GitHub Flow (deployable main, PRs) and Git flow models.
36. **Hands-on Lab:** Fork a repository, clone it, create a branch, and submit a PR back to original.
37. **Mini Assignment:** Compare differences between GitHub Flow and GitLab Flow.
38. **Common Mistakes:** Bypassing CI by pushing directly to `main`.
39. **Interview Questions:** *What is the GitHub Flow process?*
40. **Quiz Questions:** *Scenario-based: decide branching strategy*.

41. **Lesson 4.6: Git Best Practices** (Intermediate)
42. **Learning Outcome:** Apply best practices: small commits, meaningful messages, code reviews, .gitignore.
43. **Topics Covered:** Commit message conventions, .gitignore usage, avoiding large binary commits, pull request etiquette.
44. **Hands-on Lab:** Review a team's commit history to improve messages.
45. **Mini Assignment:** Write a sample commit message following guidelines (subject line + body).
46. **Common Mistakes:** Large monolithic commits; poor messages like "fix".
47. **Interview Questions:** *What makes a good Git commit message?*
48. **Quiz Questions:** *True/False on one change per commit rule.*
49. **Lesson 4.7: Advanced Git Concepts** (Advanced)
50. **Learning Outcome:** Handle advanced tasks: submodules, tags, Git bisect.
51. **Topics Covered:** `git tag` for releases, using `git bisect` to find regressions, Git submodules for embedded repos.
52. **Hands-on Lab:** Tag a release commit and find a bug using git bisect on a binary search.
53. **Mini Assignment:** Add a Git submodule to a project and update it.
54. **Common Mistakes:** Mismanaging submodule paths; forgetting to push tags.
55. **Interview Questions:** *How do you tag a commit in Git?*
56. **Quiz Questions:** *Multiple choice on git tag and bisect usage.*

Chapter 5: Docker and Containerization

Learning Objective: Teach container concepts and Docker usage for packaging applications.
Skills Covered: Containers vs VMs, Docker CLI, Dockerfiles, images, Docker Compose, volumes, networks, registry, security.
Prerequisites: Basic Linux commands, application code (e.g., a sample app).
Estimated Time: 6–8 hours
Number of Lessons: 8

1. **Lesson 5.1: Containers vs Virtual Machines** (Beginner)
2. **Learning Outcome:** Explain containerization benefits over traditional VMs【76†L54-L62】【76†L92-L100】.
3. **Topics Covered:** OS-level virtualization, isolation, images, resource efficiency【76†L54-L62】【76†L98-L101】.

4. **Hands-on Lab:** Run a simple container and inspect processes (`docker run ubuntu ps`).
5. **Mini Assignment:** Compare boot time and resource usage of a VM vs a container.
6. **Common Mistakes:** Thinking containers replace OS; not considering security implications.
7. **Interview Questions:** *Why use containers instead of VMs?*
8. **Quiz Questions:** *True/False: Containers include the OS kernel.*

9. **Lesson 5.2: Docker Installation & CLI** (Beginner)

10. **Learning Outcome:** Install Docker Engine and use basic commands.

11. **Topics Covered:** `docker pull`, `run`, `ps`, `images`, `exec`, `stop` commands.
12. **Hands-on Lab:** Install Docker on a Linux VM; pull and run `nginx` container.
13. **Mini Assignment:** Use `docker run` to start a container in detached mode and view its logs.
14. **Common Mistakes:** Running containers without resource limits; not using tags (defaults to latest).
15. **Interview Questions:** *How do you list all running Docker containers?*
16. **Quiz Questions:** *What does `docker run -it` do?*

17. **Lesson 5.3: Docker Images and Containers** (Intermediate)

18. **Learning Outcome:** Understand the relationship between images and containers.

19. **Topics Covered:** Image layering, `docker commit` vs Dockerfile, read-only images vs writable container.
20. **Hands-on Lab:** Run a container, make changes, and commit to a new image.
21. **Mini Assignment:** Use `docker history` to examine an image's layers.
22. **Common Mistakes:** Modifying containers without saving as new image; large images due to unnecessary layers.
23. **Interview Questions:** *How are Docker images versioned?*
24. **Quiz Questions:** *MCQ on image vs container differences.*

25. **Lesson 5.4: Writing Dockerfiles** (Intermediate)

26. **Learning Outcome:** Create Dockerfiles to build custom images【78†L124-L132】.

27. **Topics Covered:** Dockerfile syntax (`FROM`, `RUN`, `COPY`, `CMD` etc.)【78†L124-L132】, best practices (small layers).
28. **Hands-on Lab:** Write a Dockerfile for a simple Node.js or Python app and build the image.
29. **Mini Assignment:** Use `docker build --no-cache` and analyze build output.

30. **Common Mistakes:** Not ordering commands efficiently (caching layers); exposing secrets in images.
31. **Interview Questions:** *What's the difference between CMD and ENTRYPOINT?*
32. **Quiz Questions:** *True/False on Dockerfile commands.*

33. **Lesson 5.5: Docker Compose** (Intermediate)

34. **Learning Outcome:** Define multi-container apps using Compose files【80†L1075-L1083】.

35. **Topics Covered:** `docker-compose.yml` format: services, networks, volumes; `docker-compose up/down`【80†L1075-L1083】.
36. **Hands-on Lab:** Create a Compose file to run a web app with a database service.
37. **Mini Assignment:** Scale a service to 3 replicas using Compose (with `--scale`).
38. **Common Mistakes:** Not versioning the Compose file; not removing volumes on down.
39. **Interview Questions:** *How does Docker Compose differ from just multiple docker run commands?*
40. **Quiz Questions:** *MCQ on Docker Compose file structure.*

41. **Lesson 5.6: Docker Volumes and Networks** (Intermediate)

42. **Learning Outcome:** Persist data with volumes and connect containers with networks.

43. **Topics Covered:** Named volumes vs bind mounts, creating networks (`bridge`, `host`), port mapping (`-p`).
44. **Hands-on Lab:** Launch a container with a persistent volume (e.g., MySQL) and inspect it on host.
45. **Mini Assignment:** Connect two containers on a user-defined network and have them communicate by name.
46. **Common Mistakes:** Storing data in container file system without volumes; exposing unneeded ports.
47. **Interview Questions:** *Why use Docker volumes?*
48. **Quiz Questions:** *True/False on differences between host and bridge networks.*

49. **Lesson 5.7: Multi-Stage Docker Builds** (Advanced)

50. **Learning Outcome:** Optimize images by separating build and runtime stages【83†L412-L419】.

51. **Topics Covered:** `FROM ... AS builder`, copying artifacts between stages; reducing final image size【83†L412-L419】.
52. **Hands-on Lab:** Build a multi-stage Dockerfile for a Go or Java app (compile then package).

53. **Mini Assignment:** Compare image sizes with and without multi-stage.
54. **Common Mistakes:** Forgetting to use the final stage artifacts only; large intermediary images.
55. **Interview Questions:** *Explain benefits of multi-stage builds*.
56. **Quiz Questions:** *Multiple choice on multi-stage syntax*.

57. **Lesson 5.8: Docker Registry and Security** (Advanced)

58. **Learning Outcome:** Use Docker registries and enforce image security.

59. **Topics Covered:** Docker Hub vs private registry, `docker push/pull`, image tagging, basic image scanning.
60. **Hands-on Lab:** Push a custom image to Docker Hub or local registry, then pull on another machine.
61. **Mini Assignment:** Scan a Docker image for vulnerabilities using a tool like `docker scan` or Trivy.
62. **Common Mistakes:** Pushing untagged images (latest) without versioning; using root in containers.
63. **Interview Questions:** *How do you secure Docker images and containers?*
64. **Quiz Questions:** *True/False on using least-privilege user in Dockerfile*.

Chapter 6: Kubernetes (K8s) Fundamentals

Learning Objective: Learn how to deploy, scale, and manage containerized apps using Kubernetes.
Skills Covered: Kubernetes architecture, Pods, Deployments, ReplicaSets, Services, ConfigMaps, Secrets, Namespaces, Ingress, Volumes, StatefulSets, Helm.
Prerequisites: Docker and YAML basics.
Estimated Time: 10–12 hours
Number of Lessons: 10

1. **Lesson 6.1: Introduction to Kubernetes** (Beginner)

2. **Learning Outcome:** Describe K8s concepts: cluster, nodes, control plane, API server.

3. **Topics Covered:** Kubernetes architecture, master vs worker, `kubectl`, pods as smallest unit.
4. **Hands-on Lab:** Set up a local single-node K8s cluster with Minikube or kind.
5. **Mini Assignment:** Explore cluster resources with `kubectl get nodes` and `kubectl cluster-info`.
6. **Common Mistakes:** Not specifying `kubectl config use-context`; mixing cluster and local.
7. **Interview Questions:** *What is a Kubernetes Pod?*
8. **Quiz Questions:** *MCQ on K8s components (etcd, kubelet, etc)*.

9. **Lesson 6.2: Pods and ReplicaSets** (Intermediate)
10. **Learning Outcome:** Run containers in Pods and ensure availability with ReplicaSets.
11. **Topics Covered:** Pod spec YAML (containers, labels, ports), `kubectl run`, ReplicaSet scale.
12. **Hands-on Lab:** Create a Pod running Nginx, then convert it to a ReplicaSet of 3 replicas.
13. **Mini Assignment:** Manually kill a Pod and watch ReplicaSet recreate it.
14. **Common Mistakes:** Editing pods directly instead of using controllers; labels mismatch.
15. **Interview Questions:** *How does ReplicaSet maintain desired Pod count?*
16. **Quiz Questions:** *True/False on Pods being ephemeral.*
17. **Lesson 6.3: Deployments** (Intermediate)
18. **Learning Outcome:** Use Deployments to manage rolling updates of applications.
19. **Topics Covered:** Deployment YAML (`replicas`, `strategy`), `kubectl apply`, rolling update and rollback.
20. **Hands-on Lab:** Deploy an app via a Deployment, perform a version update, and observe rollout.
21. **Mini Assignment:** Roll back a failed Deployment to a previous revision.
22. **Common Mistakes:** Not using readiness probes (causing downtime).
23. **Interview Questions:** *What is a Deployment's purpose in K8s?*
24. **Quiz Questions:** *Multiple choice on rolling update vs recreate strategy*.
25. **Lesson 6.4: Services and Networking** (Intermediate)
26. **Learning Outcome:** Expose Pods internally and externally using Services.
27. **Topics Covered:** ClusterIP, NodePort, LoadBalancer services, service selectors, DNS in cluster.
28. **Hands-on Lab:** Create a ClusterIP service and curl it from another Pod; create a NodePort and access externally.
29. **Mini Assignment:** Simulate an external LoadBalancer using a cloud or MetalLB in local.
30. **Common Mistakes:** Service selector labels not matching Pod labels.
31. **Interview Questions:** *How does a Kubernetes Service work?*
32. **Quiz Questions:** *True/False on Service load balancing behavior*.
33. **Lesson 6.5: ConfigMaps and Secrets** (Intermediate)
34. **Learning Outcome:** Externalize configuration and sensitive data from Pods.

35. **Topics Covered:** `kubectl create configmap/secret`, mounting as env vars or files, yaml references.
36. **Hands-on Lab:** Create a ConfigMap for an app's config, and a Secret for a DB password; consume them in a Pod.
37. **Mini Assignment:** Update a ConfigMap and roll Pods to pick up new values.
38. **Common Mistakes:** Committing secrets to repos; forgetting to base64 encode secret data.
39. **Interview Questions:** *When would you use a Secret vs a ConfigMap?*
40. **Quiz Questions:** *MCQ on ConfigMap vs Secret storage.*

41. **Lesson 6.6: Namespaces** (Beginner)

42. **Learning Outcome:** Isolate resources by namespace for multi-tenancy.

43. **Topics Covered:** `kubectl create namespace`, using `-n`, context and resource quotas.
44. **Hands-on Lab:** Create two namespaces (dev, prod) and deploy same app in each without conflict.
45. **Mini Assignment:** Set a resource quota in one namespace.
46. **Common Mistakes:** Forgetting to specify namespace, leading to operations in default.
47. **Interview Questions:** *Why use namespaces in Kubernetes?*
48. **Quiz Questions:** *True/False on namespace isolation.*

49. **Lesson 6.7: Ingress Controllers** (Intermediate)

50. **Learning Outcome:** Configure Ingress to route external traffic to services.

51. **Topics Covered:** Ingress resources, Ingress controllers (NGINX, Traefik), host/path rules, TLS.
52. **Hands-on Lab:** Install an NGINX Ingress Controller and create an Ingress to a test service with a host rule.
53. **Mini Assignment:** Secure an Ingress using a self-signed TLS certificate.
54. **Common Mistakes:** Missing ingress controller; incorrect hostnames.
55. **Interview Questions:** *What is an Ingress in Kubernetes?*
56. **Quiz Questions:** *Multiple choice on Ingress object fields.*

57. **Lesson 6.8: Persistent Volumes and StatefulSets** (Advanced)

58. **Learning Outcome:** Provide stable storage and identities for stateful apps.

59. **Topics Covered:** PersistentVolume (PV) & PersistentVolumeClaim (PVC), StatefulSet for databases, volumeClaimTemplates.
60. **Hands-on Lab:** Deploy a StatefulSet (e.g., MongoDB) with a PVC per replica.
61. **Mini Assignment:** Scale a StatefulSet and observe volume creation per pod.

62. **Common Mistakes:** Using Deployment for stateful apps; not understanding volume recycling.
63. **Interview Questions:** *How does a StatefulSet differ from a Deployment?*
64. **Quiz Questions:** *True/False on PVC lifecycle tied to Pod.*

65. **Lesson 6.9: Helm Basics** (Beginner)

66. **Learning Outcome:** Use Helm to package and deploy Kubernetes applications.

67. **Topics Covered:** Helm charts, repositories, `helm install/upgrade`, templating.
68. **Hands-on Lab:** Install a Helm chart (e.g., nginx or prometheus) and customize a value.
69. **Mini Assignment:** Create a simple Helm chart for an existing Deployment.
70. **Common Mistakes:** Not versioning chart values; editing cluster resources manually instead of via Helm.
71. **Interview Questions:** *What are the benefits of using Helm?*
72. **Quiz Questions:** *Fill-in: Helm's templating language is built on ____.*

73. **Lesson 6.10: Advanced Kubernetes Concepts** (Advanced)

    - **Learning Outcome:** Explore advanced topics: CRDs and Operators.
    - **Topics Covered:** Custom Resource Definitions, Operators (e.g., for DB), Kubernetes API extensions.
    - **Hands-on Lab:** (Optional) Install a sample operator (e.g., Prometheus Operator) and create a custom resource.
    - **Mini Assignment:** List at least two use cases for custom controllers.
    - **Common Mistakes:** Overusing CRDs for simple tasks; not understanding CRD lifecycle.
    - **Interview Questions:** *What is a Kubernetes Operator?*
    - **Quiz Questions:** *MCQ on CRD vs built-in resources.*

Chapter 7: CI/CD Concepts

Learning Objective: Understand continuous integration and delivery principles and pipeline design.
Skills Covered: CI/CD pipelines, testing automation, delivery vs deployment.
Prerequisites: Git, basic Docker/Kubernetes knowledge.
Estimated Time: 3–4 hours
Number of Lessons: 4

1. **Lesson 7.1: Continuous Integration (CI)** (Beginner)

2. **Learning Outcome:** Explain CI principles: frequent code integration, automated builds and tests.

3. **Topics Covered:** Build automation, unit testing in CI pipeline, merge triggers.
4. **Hands-on Lab:** Set up a GitHub repo with a simple project and configure a CI action to run unit tests on push.
5. **Mini Assignment:** Add a failing test to see CI block merges.
6. **Common Mistakes:** Running only manual tests; infrequent integration causing big merge conflicts.
7. **Interview Questions:** *Why is continuous integration important?*
8. **Quiz Questions:** *True/False on CI reducing integration headaches*.

9. **Lesson 7.2: Continuous Delivery vs Deployment** (Beginner)

10. **Learning Outcome:** Differentiate between continuous delivery and continuous deployment.

11. **Topics Covered:** Manual vs automated deployment gates, staging environments.
12. **Hands-on Lab:** Simulate a pipeline step with manual approval before production.
13. **Mini Assignment:** Identify a manual approval step in an existing pipeline.
14. **Common Mistakes:** Automating deployment without rollback plan.
15. **Interview Questions:** *What is the difference between CD and "continuous deployment"?*
16. **Quiz Questions:** *MCQ on stages in CI/CD*.

17. **Lesson 7.3: Pipeline Design Best Practices** (Intermediate)

18. **Learning Outcome:** Design robust CI/CD pipelines (parallel tests, caching, notifications).

19. **Topics Covered:** Pipeline stages (build, test, deploy), artifact handling, retry on failure, pipeline as code.
20. **Hands-on Lab:** Sketch a pipeline flow diagram for a web app (build, test, deploy to staging, deploy to prod).
21. **Mini Assignment:** Implement caching in a CI pipeline to speed up builds.
22. **Common Mistakes:** Long-running pipelines; not failing fast on errors.
23. **Interview Questions:** *How do you handle secrets in a CI/CD pipeline?*
24. **Quiz Questions:** *Scenario: identify inefficiency in a pipeline design*.

25. **Lesson 7.4: Testing in CI/CD** (Intermediate)

26. **Learning Outcome:** Integrate automated tests (unit, integration) into pipelines.

27. **Topics Covered:** Test pyramid, regression testing, test reports, code coverage.
28. **Hands-on Lab:** Add integration tests that spin up containers (e.g., using Docker Compose) in CI.
29. **Mini Assignment:** Ensure failing tests block deployment in your CI pipeline.

30. **Common Mistakes:** Skipping tests in pipeline to save time.
31. **Interview Questions:** *What types of tests should CI run?*
32. **Quiz Questions:** *MCQ on types of automated tests*.

Chapter 8: Jenkins CI/CD

Learning Objective: Learn Jenkins for building CI/CD pipelines.
Skills Covered: Jenkins installation, pipelines (Scripted and Declarative), agents, shared libraries, GitHub integration, deployment pipelines.
Prerequisites: Java installed, basic CI/CD concepts.
Estimated Time: 6–7 hours
Number of Lessons: 6

1. **Lesson 8.1: Jenkins Installation and Setup** (Beginner)
2. **Learning Outcome:** Install Jenkins and configure basic security (admin password, plugins).
3. **Topics Covered:** Jenkins on Linux (WAR, package), installing common plugins (Git, Pipeline).
4. **Hands-on Lab:** Install Jenkins, unlock it with initial admin password, install Git and Pipeline plugins.
5. **Mini Assignment:** Change Jenkins default port and restart.
6. **Common Mistakes:** Running Jenkins with default settings (security risk).
7. **Interview Questions:** *How do you secure a new Jenkins instance?*
8. **Quiz Questions:** *True/False on Jenkins requiring a Java runtime*.
9. **Lesson 8.2: Jenkins Freestyle and Declarative Pipelines** (Intermediate)
10. **Learning Outcome:** Create jobs using freestyle projects and Jenkinsfile pipelines.
11. **Topics Covered:** Freestyle vs Pipeline jobs, syntax of Declarative Jenkinsfile (stages, steps), SCM integration.
12. **Hands-on Lab:** Build a simple Hello World Jenkinsfile with checkout, build, and test stages.
13. **Mini Assignment:** Convert an existing freestyle job into a Pipeline with a Jenkinsfile.
14. **Common Mistakes:** Mixing declarative and scripted incorrectly; hardcoding credentials.
15. **Interview Questions:** *What is a Jenkinsfile and why use it?*
16. **Quiz Questions:** *Identify correct syntax for a pipeline stage*.
17. **Lesson 8.3: Jenkins Agents and Distributed Builds** (Intermediate)
18. **Learning Outcome:** Use Jenkins agents (nodes) to distribute workloads.

19. **Topics Covered:** Master vs agent, setting up SSH or Docker agents, label assignment.
20. **Hands-on Lab:** Configure a new agent node and run a pipeline stage on it.
21. **Mini Assignment:** Label an agent and restrict a job to that label.
22. **Common Mistakes:** Overloading master with build tasks; agents not properly authenticated.
23. **Interview Questions:** *How do you add a new build agent in Jenkins?*
24. **Quiz Questions:** *MCQ on agent launching methods (JNLP, SSH, Docker)*.

25. **Lesson 8.4: Shared Libraries in Jenkins** (Advanced)

26. **Learning Outcome:** Create reusable pipelines with Shared Libraries.

27. **Topics Covered:** Jenkinsfile libraries in Git, `@Library` import, global vars and steps.
28. **Hands-on Lab:** Develop a simple shared library for common functions (e.g., notification).
29. **Mini Assignment:** Refactor duplicate pipeline code into a shared library.
30. **Common Mistakes:** Not version-controlling libraries; making libraries too broad.
31. **Interview Questions:** *What are the benefits of Jenkins shared libraries?*
32. **Quiz Questions:** *True/False on location of shared library in Jenkins*.

33. **Lesson 8.5: GitHub Integration in Jenkins** (Intermediate)

34. **Learning Outcome:** Integrate Jenkins with GitHub repositories and webhooks.

35. **Topics Covered:** GitHub plugin, GitHub Organization folder, Webhook triggers (`GitHub Hook`).
36. **Hands-on Lab:** Set up a Jenkins job to build on every GitHub push using webhooks.
37. **Mini Assignment:** Create a multibranch pipeline for a GitHub repo.
38. **Common Mistakes:** Forgetting to configure GitHub webhook URL or credentials.
39. **Interview Questions:** *How do you trigger Jenkins builds from GitHub?*
40. **Quiz Questions:** *Multiple choice on Jenkins GitHub plugin features*.

41. **Lesson 8.6: Building Deployment Pipelines with Jenkins** (Intermediate)

42. **Learning Outcome:** Orchestrate end-to-end CI/CD pipelines in Jenkins (build, test, deploy).

43. **Topics Covered:** Pipeline as code for full flow, environment promotion, Blue/Green or Canary via Jenkins.
44. **Hands-on Lab:** Create a multi-stage pipeline that builds a Docker image, runs tests, and pushes to a registry.
45. **Mini Assignment:** Add a stage to deploy a successful build to a Kubernetes cluster.
46. **Common Mistakes:** Blocking stages for manual approval without notification.

47. **Interview Questions:** *How do you implement approvals in a Jenkins pipeline?*
48. **Quiz Questions:** *Scenario-based: identify a failed step in a pipeline script.*

Chapter 9: GitHub Actions CI/CD

Learning Objective: Learn to implement CI/CD using GitHub Actions.
Skills Covered: Workflow files, actions, building/testing/deploy pipelines on GitHub.
Prerequisites: GitHub account, repository access.
Estimated Time: 4–5 hours
Number of Lessons: 5

1. **Lesson 9.1: GitHub Actions Basics** (Beginner)
2. **Learning Outcome:** Create a basic GitHub Action workflow file.
3. **Topics Covered:** `workflow.yml` syntax, triggers (`on: push,pull_request`), jobs and steps.
4. **Hands-on Lab:** Add a workflow that runs on every push and echoes "Hello World".
5. **Mini Assignment:** Test workflow by pushing a commit; view action logs.
6. **Common Mistakes:** YAML syntax errors, forgetting `actions/checkout@v2`.
7. **Interview Questions:** *What is the purpose of `actions/checkout` in a workflow?*
8. **Quiz Questions:** *True/False on file location (.github/workflows).*
9. **Lesson 9.2: Build and Test Pipeline** (Intermediate)
10. **Learning Outcome:** Implement build and test stages in Actions.
11. **Topics Covered:** Setting up environment (e.g., Node, Java), running tests, uploading artifacts.
12. **Hands-on Lab:** Configure Actions to build a sample app and run its test suite.
13. **Mini Assignment:** Fail the build if tests fail; examine build badge update.
14. **Common Mistakes:** Not caching dependencies (long build times).
15. **Interview Questions:** *How do you add multiple steps in a job?*
16. **Quiz Questions:** *MCQ on `runs-on` runners (ubuntu-latest, windows, etc.).*
17. **Lesson 9.3: Deployment Pipeline** (Intermediate)
18. **Learning Outcome:** Automate deployment to an environment (e.g., AWS, Azure, K8s) using Actions.
19. **Topics Covered:** `actions/upload-artifact`, `actions/deploy`, environment secrets, manual approval (`environment: production`).
20. **Hands-on Lab:** Use a GitHub Action to deploy a Docker image to Docker Hub or AWS ECR.
21. **Mini Assignment:** Add a conditional (branch or tag) to deploy only on `main` pushes.

22. **Common Mistakes:** Committing secrets instead of using GitHub Secrets.
23. **Interview Questions:** *How do GitHub Environments and secrets work?*
24. **Quiz Questions:** *True/False on using Actions to deploy to Kubernetes*.

25. **Lesson 9.4: Marketplace Actions and Custom Actions** (Advanced)

26. **Learning Outcome:** Utilize pre-built actions and create custom ones.

27. **Topics Covered:** GitHub Actions marketplace, composite actions, action development (Docker or JS action).
28. **Hands-on Lab:** Use an official action (e.g., GitHub Pages deploy) in a workflow.
29. **Mini Assignment:** Write a simple composite action to run a set of commands.
30. **Common Mistakes:** Using outdated or unmaintained marketplace actions.
31. **Interview Questions:** *What are the types of GitHub Actions you can create?*
32. **Quiz Questions:** *MCQ on differences between composite and JavaScript actions*.

33. **Lesson 9.5: GitHub Actions Best Practices** (Intermediate)

34. **Learning Outcome:** Follow best practices: small jobs, caching, artifact retention, YAML modularity.

35. **Topics Covered:** Workflow templates, matrix builds for multi-OS, using cache, action reusability.
36. **Hands-on Lab:** Convert repetitive steps into a reusable workflow file.
37. **Mini Assignment:** Implement a build matrix for Node.js versions in your workflow.
38. **Common Mistakes:** Ignoring concurrency (not canceling outdated runs).
39. **Interview Questions:** *How do you reduce redundant runs in GitHub Actions?*
40. **Quiz Questions:** *True/False on parallel vs sequential job execution*.

Chapter 10: Infrastructure as Code (Terraform)

Learning Objective: Use Terraform to provision and manage cloud infrastructure declaratively.
Skills Covered: Terraform CLI (init, plan, apply), HCL, variables, modules, remote state, workspaces.
Prerequisites: Basic Linux CLI, AWS account (for AWS examples).
Estimated Time: 5–6 hours
Number of Lessons: 6

1. **Lesson 10.1: Terraform Introduction and Setup** (Beginner)

2. **Learning Outcome:** Install Terraform and understand IaC concepts【90†L110-L118】.

3. **Topics Covered:** Terraform installation, configuration files (.tf), Terraform providers, init-plan-apply lifecycle.

4. **Hands-on Lab:** Write a simple Terraform config to launch one AWS EC2 instance.
5. **Mini Assignment:** Preview and apply the plan to create a resource.
6. **Common Mistakes:** Applying without review; not storing state in version control.
7. **Interview Questions:** *What is Infrastructure as Code and why use Terraform?*
8. **Quiz Questions:** *True/False on Terraform storing state locally by default.*

9. **Lesson 10.2: Variables and Outputs** (Intermediate)

10. **Learning Outcome:** Use variables for dynamic configs and outputs for values【90†L116-L124】.

11. **Topics Covered:** `variable` blocks, `terraform.tfvars`, `output` blocks, sensitive variables.
12. **Hands-on Lab:** Add variables for instance type and AMI in your AWS example.
13. **Mini Assignment:** Output the public IP of the created instance.
14. **Common Mistakes:** Hardcoding values, forgetting to handle secrets.
15. **Interview Questions:** *How do you pass variables to Terraform?*
16. **Quiz Questions:** *MCQ on default variable values and environment variables.*

17. **Lesson 10.3: Terraform Modules** (Intermediate)

18. **Learning Outcome:** Organize Terraform config into reusable modules.

19. **Topics Covered:** Module structure, `module` block usage, versioning modules (local vs remote).
20. **Hands-on Lab:** Create a module for an AWS network (VPC, subnets) and reuse it.
21. **Mini Assignment:** Refactor duplicate resources into a module.
22. **Common Mistakes:** Over-complicating modules; not using source control for modules.
23. **Interview Questions:** *What are Terraform modules and why use them?*
24. **Quiz Questions:** *True/False: Modules cannot call other modules.*

25. **Lesson 10.4: Remote State and Workspaces** (Intermediate)

26. **Learning Outcome:** Manage Terraform state remotely and use workspaces for environments.

27. **Topics Covered:** Backend configuration (S3, Terraform Cloud), locking (DynamoDB), workspaces (dev/prod).
28. **Hands-on Lab:** Configure an S3 backend for state and create a second workspace (e.g., “staging”).
29. **Mini Assignment:** Migrate local state to S3; switch to workspace `prod` and apply changes.
30. **Common Mistakes:** State file conflicts, not locking remote state.

31. **Interview Questions:** *Why use remote state storage?*
32. **Quiz Questions:** *MCQ on Terraform workspace isolation.*

33. **Lesson 10.5: Terraform Best Practices** (Intermediate)

34. **Learning Outcome:** Follow IaC best practices: state management, locking, version pinning.

35. **Topics Covered:** Code organization (one resource per file), using `terraform fmt`/`validate`, managing secrets.
36. **Hands-on Lab:** Format and lint your Terraform code; lock provider versions.
37. **Mini Assignment:** Plan a change and review terraform plan output carefully.
38. **Common Mistakes:** Not reviewing `plan` before apply; losing track of resource drift.
39. **Interview Questions:** *How do you enforce policies in Terraform usage?*
40. **Quiz Questions:** *True/False on importance of versioning the state file.*

41. **Lesson 10.6: Advanced Terraform – Provisioners & Cloud Init** (Advanced)

42. **Learning Outcome:** Use provisioners and cloud-init for post-provisioning tasks.

43. **Topics Covered:** `provisioner "remote-exec"`, `null_resource` triggers, data sources, cloud-init user data for instances.
44. **Hands-on Lab:** Provision a web server that installs Nginx via remote-exec after create.
45. **Mini Assignment:** Use `local-exec` to generate a config file before applying resources.
46. **Common Mistakes:** Relying too much on provisioners instead of immutable infrastructure.
47. **Interview Questions:** *When should you use Terraform provisioners?*
48. **Quiz Questions:** *Multiple choice on data sources vs resources.*

Chapter 11: Configuration Management (Ansible)

Learning Objective: Automate server configuration using Ansible.
Skills Covered: Ansible installation, inventory management, playbooks, roles, variables, templates, best practices.
Prerequisites: SSH access to servers, basic Linux admin.
Estimated Time: 5–6 hours
Number of Lessons: 7

1. **Lesson 11.1: Ansible Basics and Installation** (Beginner)

2. **Learning Outcome:** Install Ansible and run ad-hoc commands on inventory.

3. **Topics Covered:** Agentless architecture, inventory file (`hosts`), `ansible -m ping`, Python requirement on hosts.

4. **Hands-on Lab:** Install Ansible on control machine; ping multiple Linux hosts.
5. **Mini Assignment:** Use `ansible -m shell -a "uname -a"` on all hosts.
6. **Common Mistakes:** Forgetting SSH keys or missing Python on target hosts.
7. **Interview Questions:** *How does Ansible connect to target machines?*
8. **Quiz Questions:** *True/False: Ansible requires an agent on remote nodes*.

9. **Lesson 11.2: Ansible Inventory and Configuration** (Beginner)

10. **Learning Outcome:** Organize hosts into groups and configure ansible.cfg.

11. **Topics Covered:** Static vs dynamic inventory, group_vars, host_vars, INI vs YAML inventory.
12. **Hands-on Lab:** Create an inventory with groups (web, db) and test group-specific ping.
13. **Mini Assignment:** Add a new host under `db` group and run tasks only on `db`.
14. **Common Mistakes:** Misnaming groups or hosts; forgetting to specify inventory file.
15. **Interview Questions:** *What is Ansible inventory?*
16. **Quiz Questions:** *MCQ on inventory syntax*.

17. **Lesson 11.3: Ansible Playbooks and Ad-Hoc** (Intermediate)

18. **Learning Outcome:** Write playbooks to automate tasks (beyond ad-hoc).

19. **Topics Covered:** Playbook structure (`hosts`, `tasks`), YAML syntax, idempotence.
20. **Hands-on Lab:** Create a playbook to install and start Nginx on web servers.
21. **Mini Assignment:** Run playbook with `--check` (dry-run) and `--diff`.
22. **Common Mistakes:** Not using `become` for sudo tasks; running commands instead of modules.
23. **Interview Questions:** *What is idempotence in Ansible?*
24. **Quiz Questions:** *True/False: Ansible playbooks are just YAML lists of tasks*.

25. **Lesson 11.4: Tasks, Modules, and Handlers** (Intermediate)

26. **Learning Outcome:** Use Ansible modules and handlers for service changes.

27. **Topics Covered:** Common modules (`yum`/`apt`, `copy`, `service`, `file`), `notify` handlers.
28. **Hands-on Lab:** Write tasks to deploy a config file and restart service only if changed (using handler).
29. **Mini Assignment:** Use `copy` module with `template` instead of raw copy.
30. **Common Mistakes:** Using `command` module for tasks better handled by specific modules.
31. **Interview Questions:** *When do you use handlers in Ansible?*
32. **Quiz Questions:** *MCQ on purpose of handlers*.

33. **Lesson 11.5: Roles and Playbook Organization** (Intermediate)
34. **Learning Outcome:** Structure playbooks into roles for reusability.
35. **Topics Covered:** `ansible-galaxy init`, role directory layout (tasks/, handlers/, templates/), including roles in playbooks.
36. **Hands-on Lab:** Refactor Nginx setup into a `nginx` role and call it from a play.
37. **Mini Assignment:** Share a role between two projects by placing it in `roles/`.
38. **Common Mistakes:** Flat playbooks becoming unwieldy; not using `defaults` vs `vars`.
39. **Interview Questions:** *How do roles improve playbook maintenance?*
40. **Quiz Questions:** *True/False: Roles have fixed directory names like `tasks/main.yml`.*
41. **Lesson 11.6: Variables and Templates** (Intermediate)
42. **Learning Outcome:** Use variables for dynamic configuration and Jinja2 templates.
43. **Topics Covered:** Variable precedence, host_vars/group_vars, `vars_prompt`, `with_items`, templating config files.
44. **Hands-on Lab:** Use a Jinja2 template for an Nginx vhost file with host-specific values.
45. **Mini Assignment:** Encrypt a variable (using Ansible Vault) and use it in a play.
46. **Common Mistakes:** Variable name conflicts; cleartext secrets in playbooks.
47. **Interview Questions:** *What is Ansible Vault used for?*
48. **Quiz Questions:** *MCQ on variable precedence (defaults < vars < extra-vars).*
49. **Lesson 11.7: Ansible Best Practices and Galaxy** (Intermediate)
50. **Learning Outcome:** Follow Ansible best practices and use Ansible Galaxy roles.
51. **Topics Covered:** Directory layout (playbooks/, roles/, inventories/), using community roles from Galaxy, version control.
52. **Hands-on Lab:** Install a role from Ansible Galaxy (e.g., `geerlingguy.mysql`) and use it.
53. **Mini Assignment:** Lint a playbook with `ansible-lint` and fix issues.
54. **Common Mistakes:** Ignoring Ansible YAML errors; re-inventing the wheel for common tasks.
55. **Interview Questions:** *How do you use Ansible Galaxy roles?*
56. **Quiz Questions:** *True/False on storing playbooks in version control.*

Chapter 12: Cloud Computing Fundamentals (AWS Focus)

Learning Objective: Introduce cloud concepts and AWS core services.
Skills Covered: AWS account setup, IAM, EC2, S3, VPC, RDS, Route 53, CloudWatch, ECR/ECS/EKS basics, high-level cloud design.

Prerequisites: AWS free tier account (recommended).
Estimated Time: 10–12 hours
Number of Lessons: 10

1. **Lesson 12.1: Cloud Computing Concepts & AWS Overview** (Beginner)
2. **Learning Outcome:** Understand cloud models (IaaS, PaaS, SaaS) and AWS global infrastructure.
3. **Topics Covered:** Regions vs AZs, shared responsibility model, basic AWS console navigation.
4. **Hands-on Lab:** Explore AWS Management Console; find and copy access keys for CLI.
5. **Mini Assignment:** Set up AWS CLI with your account.
6. **Common Mistakes:** Using root credentials for daily tasks; not enabling billing alerts.
7. **Interview Questions:** *What is AWS?*
8. **Quiz Questions:** *MCQ on types of cloud services (IaaS vs PaaS).*
9. **Lesson 12.2: AWS IAM (Identity and Access Management)** (Intermediate)
10. **Learning Outcome:** Manage users, groups, roles and policies in AWS.
11. **Topics Covered:** Creating IAM users with least privilege, IAM roles and policies, MFA setup.
12. **Hands-on Lab:** Create a new IAM user with programmatic access and assign to an Admin group.
13. **Mini Assignment:** Implement MFA on the root account and your new user.
14. **Common Mistakes:** Overly broad permissions; using root account.
15. **Interview Questions:** *Why is IAM important in AWS?*
16. **Quiz Questions:** *True/False: IAM Roles are only for EC2 instances.*
17. **Lesson 12.3: EC2 Virtual Servers** (Intermediate)
18. **Learning Outcome:** Launch and configure EC2 instances【98†L12-L20】.
19. **Topics Covered:** Instance types, AMIs, key pairs, security groups, EBS volumes.
20. **Hands-on Lab:** Launch an EC2 instance with a public IP, SSH into it, install a web server.
21. **Mini Assignment:** Create a custom AMI from a configured instance.
22. **Common Mistakes:** Leaving SSH port open to 0.0.0.0/0; forgetting to assign an IAM role for EC2.
23. **Interview Questions:** *What is an AMI?*
24. **Quiz Questions:** *MCQ on EC2 pricing models (On-Demand, Spot, Reserved).*

25. **Lesson 12.4: Amazon S3 Storage** (Intermediate)
26. **Learning Outcome:** Use S3 for object storage and static website hosting.
27. **Topics Covered:** Buckets, object lifecycle, versioning, permissions (bucket policies, IAM).
28. **Hands-on Lab:** Create an S3 bucket, upload files, set public read for static website.
29. **Mini Assignment:** Configure a bucket policy to restrict access to specific IPs.
30. **Common Mistakes:** Publicly exposing sensitive data; not using lifecycle rules for cleanup.
31. **Interview Questions:** *How do you secure an S3 bucket?*
32. **Quiz Questions:** *True/False on S3 durability and availability.*
33. **Lesson 12.5: AWS Networking (VPC)** (Intermediate)
34. **Learning Outcome:** Set up a basic VPC with subnets, route tables, and internet gateway.
35. **Topics Covered:** Subnetting, IGW, NAT, route tables, Security Groups vs NACLs.
36. **Hands-on Lab:** Create a new VPC with public and private subnets; launch instances in each.
37. **Mini Assignment:** Add a NAT Gateway for outbound internet from private subnet.
38. **Common Mistakes:** Misconfiguring route tables; using default VPC insecurely.
39. **Interview Questions:** *What is a VPC and why use it?*
40. **Quiz Questions:** *MCQ on differences between SG and NACL.*
41. **Lesson 12.6: AWS RDS and Databases** (Intermediate)
42. **Learning Outcome:** Launch and configure managed databases with RDS.
43. **Topics Covered:** RDS engine choices (MySQL, PostgreSQL, etc), Multi-AZ, parameter groups.
44. **Hands-on Lab:** Create an RDS MySQL instance in a private subnet and connect from EC2.
45. **Mini Assignment:** Enable automatic backups and read replica.
46. **Common Mistakes:** Exposing DB publicly; not setting master user securely.
47. **Interview Questions:** *What is Amazon RDS used for?*
48. **Quiz Questions:** *True/False on RDS automatic failover.*
49. **Lesson 12.7: AWS Route 53 (DNS Service)** (Intermediate)
50. **Learning Outcome:** Configure DNS zones and records with Route 53.
51. **Topics Covered:** Public vs private hosted zones, record types (A, CNAME, MX), health checks.

52. **Hands-on Lab:** Register a domain (or use an existing one) and point `www` to an EC2 instance.
53. **Mini Assignment:** Set up a weighted routing policy for `www` between two IPs.
54. **Common Mistakes:** Forgetting to set TTL; mixing up record types.
55. **Interview Questions:** *How do you use Route 53 for high availability?*
56. **Quiz Questions:** *MCQ on types of routing policies (geolocation, latency).*

57. **Lesson 12.8: AWS Monitoring (CloudWatch)** (Intermediate)

58. **Learning Outcome:** Monitor AWS resources using CloudWatch metrics and alarms.

59. **Topics Covered:** CloudWatch metrics (CPU, network), custom metrics, alarms, logs (CloudWatch Logs).
60. **Hands-on Lab:** Create a CloudWatch alarm to email when EC2 CPU > 70%.
61. **Mini Assignment:** Enable detailed monitoring and create a dashboard.
62. **Common Mistakes:** Not tagging resources (hard to filter), ignoring CloudWatch costs.
63. **Interview Questions:** *What is CloudWatch used for?*
64. **Quiz Questions:** *True/False on CloudWatch as a log aggregator.*

65. **Lesson 12.9: Container Services (ECR, ECS, EKS)** (Intermediate)

66. **Learning Outcome:** Learn basics of AWS container services.

67. **Topics Covered:** Amazon ECR (Docker registry), ECS (Docker orchestration), EKS (managed Kubernetes) overview.
68. **Hands-on Lab:** Push a Docker image to ECR and deploy it on ECS with Fargate launch type.
69. **Mini Assignment:** List differences between ECS and EKS.
70. **Common Mistakes:** Using EC2 launch type without ASG (scaling), not securing ECR.
71. **Interview Questions:** *What is Amazon ECS and when would you use it?*
72. **Quiz Questions:** *MCQ on EKS advantages over ECS.*

73. **Lesson 12.10: AWS Security and IAM Advanced** (Advanced)

    - **Learning Outcome:** Implement advanced AWS security: KMS, Organizations, Security Hub.
    - **Topics Covered:** AWS KMS (encryption), IAM policies (least privilege), AWS Organizations for multi-account, AWS Config.
    - **Hands-on Lab:** Create an IAM policy that restricts S3 actions and attach to a user.
    - **Mini Assignment:** Enable AWS Config to record all S3 resources.

- **Common Mistakes:** Hardcoding credentials; using wildcard actions in policies.
- **Interview Questions:** *How do you encrypt data at rest in AWS?*
- **Quiz Questions:** *True/False on cross-account IAM roles.*

Chapter 13: Monitoring & Observability

Learning Objective: Implement modern monitoring solutions for metrics and visualization.
Skills Covered: Prometheus, Grafana, alerting, metrics collection, dashboard creation.
Prerequisites: Kubernetes basics (for Prometheus integration) or Docker; some app to monitor.
Estimated Time: 5–6 hours
Number of Lessons: 5

1. **Lesson 13.1: Prometheus Monitoring** (Intermediate)
2. **Learning Outcome:** Set up Prometheus to scrape metrics from services【100†L25-L33】【100†L42-L48】.
3. **Topics Covered:** Prometheus architecture, exporters, time-series data, PromQL basics.
4. **Hands-on Lab:** Deploy Prometheus (Docker or Kubernetes) and configure node_exporter to gather host metrics.
5. **Mini Assignment:** Write a PromQL query to get CPU usage over time.
6. **Common Mistakes:** Not using labels effectively; scraping too frequently (noisy data).
7. **Interview Questions:** *What is Prometheus and why use it?*
8. **Quiz Questions:** *True/False on Prometheus pull-based scraping.*
9. **Lesson 13.2: Grafana Dashboards** (Intermediate)
10. **Learning Outcome:** Create Grafana dashboards for visualizing metrics【103†L344-L352】.
11. **Topics Covered:** Connecting data sources (Prometheus, CloudWatch), panels (graphs, singlestat), alerting.
12. **Hands-on Lab:** Add Prometheus as a data source in Grafana; build a dashboard (e.g., HTTP request count over time).
13. **Mini Assignment:** Configure an alert in Grafana on a threshold (e.g., high CPU).
14. **Common Mistakes:** Overcrowded dashboards; not sharing dashboards via JSON.
15. **Interview Questions:** *Why use Grafana with Prometheus?*
16. **Quiz Questions:** *MCQ on types of Grafana panels.*
17. **Lesson 13.3: Alerting and Notification** (Intermediate)

18. **Learning Outcome:** Set up alerts with Prometheus Alertmanager and Grafana.
19. **Topics Covered:** Alertmanager configuration (routing, receivers), annotation, notification channels (email, Slack).
20. **Hands-on Lab:** Create an alert rule in Prometheus (e.g., instance down) and route alerts to email.
21. **Mini Assignment:** Silence an alert during maintenance using Alertmanager's web UI.
22. **Common Mistakes:** Not tuning alert thresholds (too sensitive); ignoring alert fatigue.
23. **Interview Questions:** *How do you avoid alert noise in monitoring?*
24. **Quiz Questions:** *True/False on deduplication in Alertmanager.*
25. **Lesson 13.4: Metrics Exporters and Integration** (Intermediate)
26. **Learning Outcome:** Use exporters to monitor applications and infrastructure.
27. **Topics Covered:** Node exporter, cAdvisor, CloudWatch exporter, application metrics (client libraries).
28. **Hands-on Lab:** Install Node Exporter on a VM and visualize its metrics in Prometheus/Grafana.
29. **Mini Assignment:** Expose a custom app metric (e.g., request count) using Prometheus client lib.
30. **Common Mistakes:** Exposing metrics publicly; scraping insecure endpoints.
31. **Interview Questions:** *What are Prometheus exporters?*
32. **Quiz Questions:** *MCQ on examples of exporters (PostgreSQL, MySQL, etc.).*
33. **Lesson 13.5: Monitoring in Cloud Environments** (Intermediate)
34. **Learning Outcome:** Monitor cloud resources (e.g., AWS) with Prometheus or other services.
35. **Topics Covered:** CloudWatch exporter for Prometheus, Grafana Cloud, managed monitoring (e.g., DataDog).
36. **Hands-on Lab:** Configure Prometheus to scrape AWS metrics via CloudWatch exporter.
37. **Mini Assignment:** Compare native CloudWatch dashboard vs custom Grafana dashboard for the same metric.
38. **Common Mistakes:** Billing surprise from extensive monitoring; not filtering metrics (too many).
39. **Interview Questions:** *How can you monitor AWS EC2 using Prometheus?*
40. **Quiz Questions:** *True/False: Grafana Cloud is free for open-source use.*

Chapter 14: Logging and Log Management

Learning Objective: Implement centralized logging solutions for DevOps.
Skills Covered: ELK Stack (Elasticsearch, Logstash, Kibana), Grafana Loki, log shippers (Beats, Fluentd).
Prerequisites: ElasticSearch installation basics (or Docker), familiarity with logs.
Estimated Time: 5–6 hours
Number of Lessons: 5

1. **Lesson 14.1: ELK Stack Overview** (Intermediate)
2. **Learning Outcome:** Understand the components of the ELK Stack and log pipelines【105†L190-L199】.
3. **Topics Covered:** Elasticsearch, Logstash, Kibana, Beats, ingestion and indexing.
4. **Hands-on Lab:** Deploy Elasticsearch, Logstash, and Kibana (via Docker) and send sample logs to it.
5. **Mini Assignment:** Create an index in Kibana and view a log entry.
6. **Common Mistakes:** Storing logs without mapping (text vs keyword fields).
7. **Interview Questions:** *What is Elasticsearch used for?*
8. **Quiz Questions:** *True/False on Logstash requiring Kibana to function*.
9. **Lesson 14.2: Logstash and Beats** (Intermediate)
10. **Learning Outcome:** Parse and ship logs using Logstash and Beats (Filebeat).
11. **Topics Covered:** Logstash filters, pipeline config, Filebeat installation and modules.
12. **Hands-on Lab:** Configure Filebeat to send syslog or Apache logs to Logstash and Elasticsearch.
13. **Mini Assignment:** Write a Logstash grok filter to parse a custom log format.
14. **Common Mistakes:** High CPU usage from grok; forgetting to restart services after config changes.
15. **Interview Questions:** *When would you use Filebeat vs Metricbeat?*
16. **Quiz Questions:** *MCQ on Beats default modules*.
17. **Lesson 14.3: Kibana Dashboards and Search** (Intermediate)
18. **Learning Outcome:** Explore logs with Kibana and build dashboards.
19. **Topics Covered:** Searching logs (KQL), visualizations (pie charts, timelines), creating alerts (Watchers or Elastalert).
20. **Hands-on Lab:** Use Kibana Discover to find errors; build a dashboard of error count per host.

21. **Mini Assignment:** Set up a Kibana Watch (or use Elastalert) to alert on a specific log pattern.
22. **Common Mistakes:** Querying without filters (getting too much data), not using time ranges.
23. **Interview Questions:** *How do you optimize search in Elasticsearch?*
24. **Quiz Questions:** *True/False on using Kibana to create indices.*

25. **Lesson 14.4: Grafana Loki** (Intermediate)

26. **Learning Outcome:** Collect and query logs using Grafana Loki【107†L42-L49】【107†L65-L73】.

27. **Topics Covered:** Loki architecture, Promtail agent, LogQL query language.
28. **Hands-on Lab:** Deploy Grafana Loki and Promtail; send application logs and query them in Grafana.
29. **Mini Assignment:** Write a LogQL query to filter logs containing "ERROR" label from a certain container.
30. **Common Mistakes:** Treating Loki like Elasticsearch (it only indexes metadata).
31. **Interview Questions:** *What is unique about Loki's approach to log indexing?*
32. **Quiz Questions:** *MCQ on LogQL syntax.*

33. **Lesson 14.5: Centralized Logging Best Practices** (Intermediate)

34. **Learning Outcome:** Implement logging strategy: retention, parsing, monitoring logs.

35. **Topics Covered:** Log rotation and retention policies, structuring logs (JSON), privacy (masking sensitive data).
36. **Hands-on Lab:** Configure Filebeat to add fields (e.g., environment) to logs before indexing.
37. **Mini Assignment:** Plan retention (e.g., 30 days) and simulate log cleanup.
38. **Common Mistakes:** Excessive retention (cost); logging PII unencrypted.
39. **Interview Questions:** *What is log rotation and why is it important?*
40. **Quiz Questions:** *True/False on benefits of centralized logging.*

Chapter 15: Security and DevSecOps

Learning Objective: Introduce security practices within the DevOps lifecycle.
Skills Covered: DevSecOps concepts, secrets management, SSL/TLS, IAM security, vulnerability scanning, image scanning, secure coding.
Prerequisites: Basic security knowledge (TLS, SSH).
Estimated Time: 5–6 hours
Number of Lessons: 6

1. **Lesson 15.1: DevSecOps Principles** (Beginner)

2. **Learning Outcome:** Understand integrating security into DevOps (shift-left security).
3. **Topics Covered:** Secure SDLC, automated security testing, compliance as code.
4. **Hands-on Lab:** (Conceptual) Add a security lint step in CI for YAML or Dockerfile.
5. **Mini Assignment:** Identify open-source tools for security scanning (e.g., OWASP ZAP).
6. **Common Mistakes:** Treating security as separate phase at end.
7. **Interview Questions:** *What is DevSecOps?*
8. **Quiz Questions:** *True/False on scanning images before deployment.*
9. **Lesson 15.2: Secrets Management** (Intermediate)
10. **Learning Outcome:** Manage sensitive information (passwords, API keys) securely.
11. **Topics Covered:** Tools like HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets, environment variables.
12. **Hands-on Lab:** Set up Vault (dev mode) to store a secret and retrieve it via CLI.
13. **Mini Assignment:** Use Ansible Vault or sops to encrypt a config file.
14. **Common Mistakes:** Hardcoding secrets in code or config; using same credentials everywhere.
15. **Interview Questions:** *How do you handle secrets in CI/CD?*
16. **Quiz Questions:** *MCQ on difference between HashiCorp Vault and AWS Secrets Manager.*
17. **Lesson 15.3: SSL/TLS and Certificates** (Intermediate)
18. **Learning Outcome:** Configure TLS encryption for services (HTTPS).
19. **Topics Covered:** Certificate Authorities (CA), generating certs (OpenSSL), Let's Encrypt automation, TLS in Kubernetes (Ingress TLS).
20. **Hands-on Lab:** Generate a self-signed cert for Nginx, configure it; or use Certbot for a domain.
21. **Mini Assignment:** Implement automated certificate renewal.
22. **Common Mistakes:** Using weak ciphers; forgetting to secure the private key.
23. **Interview Questions:** *What is TLS handshake?*
24. **Quiz Questions:** *True/False on purpose of intermediate CAs.*
25. **Lesson 15.4: IAM and Access Control** (Intermediate)
26. **Learning Outcome:** Enforce least-privilege and role-based access.
27. **Topics Covered:** Principle of least privilege, role-based access (AWS IAM roles, GCP IAM), multi-factor authentication.

28. **Hands-on Lab:** Audit an IAM policy for excessive permissions (e.g., `*:*`) and tighten it.
29. **Mini Assignment:** Create an IAM role with permissions limited to S3 read-only.
30. **Common Mistakes:** Overprivileged service accounts; sharing keys between teams.
31. **Interview Questions:** *What is least privilege and why is it important?*
32. **Quiz Questions:** *MCQ on MFA vs password alone*.

33. **Lesson 15.5: Vulnerability and Image Scanning** (Intermediate)

34. **Learning Outcome:** Scan code, containers, and infrastructure for vulnerabilities.

35. **Topics Covered:** Tools: Snyk, Clair, Trivy for images; OWASP Dependency Check for code; automation in CI.
36. **Hands-on Lab:** Scan a Docker image with `docker scan` or Trivy and interpret results.
37. **Mini Assignment:** Integrate an image scan into your CI pipeline.
38. **Common Mistakes:** Ignoring low/medium findings; not updating dependencies.
39. **Interview Questions:** *How would you automate vulnerability scanning in DevOps?*
40. **Quiz Questions:** *True/False: Docker Hub performs free security scans*.

41. **Lesson 15.6: Security Best Practices** (Advanced)

42. **Learning Outcome:** Adopt security best practices across stack.

43. **Topics Covered:** Secure coding (OWASP Top 10), patch management, network segmentation, auditing and logging access.
44. **Hands-on Lab:** Review code for SQL injection or XSS patterns.
45. **Mini Assignment:** Apply a security patch on an outdated package.
46. **Common Mistakes:** Exposing debug info; not rotating keys or certs regularly.
47. **Interview Questions:** *What is the OWASP Top 10?*
48. **Quiz Questions:** *MCQ on examples of injection attacks*.

Chapter 16: Nginx as Reverse Proxy and Load Balancer

Learning Objective: Use Nginx for proxying, load balancing, and SSL termination.
Skills Covered: Nginx configuration, reverse proxy setup, upstream load balancing, SSL.
Prerequisites: Linux basics, networking.
Estimated Time: 2 hours
Number of Lessons: 3

1. **Lesson 16.1: Nginx Reverse Proxy** (Intermediate)

2. **Learning Outcome:** Configure Nginx as a reverse proxy for backend servers.

3. **Topics Covered:** `proxy_pass`, upstream block, headers (Host, X-Forwarded-For).

4. **Hands-on Lab:** Set up Nginx to proxy requests to a local app server running on another port.
5. **Mini Assignment:** Add health checks to the upstream if possible.
6. **Common Mistakes:** Missing `proxy_set_header`; not securing Nginx itself.
7. **Interview Questions:** *How do you configure Nginx to forward traffic to multiple app servers?*
8. **Quiz Questions:** *True/False on the role of `upstream {}` block.*

9. **Lesson 16.2: Nginx Load Balancing** (Intermediate)

10. **Learning Outcome:** Distribute requests among multiple backend servers.

11. **Topics Covered:** Load balancing methods (round robin, least_conn), sticky sessions, `max_fails`, `fail_timeout`.
12. **Hands-on Lab:** Configure an upstream with multiple servers and test load distribution.
13. **Mini Assignment:** Simulate a server failure and observe Nginx failover.
14. **Common Mistakes:** No session affinity causing issues for login-based apps.
15. **Interview Questions:** *How does Nginx handle a failing backend?*
16. **Quiz Questions:** *MCQ on difference between ip_hash and least_conn.*

17. **Lesson 16.3: SSL Termination with Nginx** (Intermediate)

18. **Learning Outcome:** Offload SSL (HTTPS) from backend services using Nginx.

19. **Topics Covered:** `listen 443 ssl`, certificate directives (`ssl_certificate`, `ssl_certificate_key`), HTTP->HTTPS redirect.
20. **Hands-on Lab:** Configure Nginx with a TLS certificate and key for your domain.
21. **Mini Assignment:** Redirect all HTTP requests to HTTPS.
22. **Common Mistakes:** Using outdated TLS versions; not enabling HSTS.
23. **Interview Questions:** *Why use Nginx for SSL termination?*
24. **Quiz Questions:** *True/False: TLS certificates can only be in `/etc/ssl/`.*

Chapter 17: Apache HTTP Server Basics

Learning Objective: Introduce Apache HTTP Server setup and configuration.
Skills Covered: Apache installation, vhosts, modules, proxying.
Prerequisites: Linux basics.
Estimated Time: 2 hours
Number of Lessons: 3

1. **Lesson 17.1: Apache Installation and Virtual Hosts** (Beginner)

2. **Learning Outcome:** Install Apache and configure virtual hosts for multiple domains.

3. **Topics Covered:** `yum/apt install httpd`, `sites-available`, `sites-enabled`, `DocumentRoot`.
4. **Hands-on Lab:** Host two local domains (e.g., example.com and test.com) on one Apache instance.
5. **Mini Assignment:** Enable SSL for a virtual host.
6. **Common Mistakes:** Mixing up IP-based vs name-based vhosts; permissions on web root.
7. **Interview Questions:** *How do you enable a virtual host in Apache?*
8. **Quiz Questions:** *MCQ on Apache directive for enabling a vhost.*
9. **Lesson 17.2: Apache Modules and Performance** (Intermediate)
10. **Learning Outcome:** Load/unload Apache modules and tune performance.
11. **Topics Covered:** Enabling modules (`mod_proxy`, `mod_ssl`), `worker` vs `prefork`, `KeepAlive`.
12. **Hands-on Lab:** Enable `mod_proxy` and proxy a backend like in Nginx lab.
13. **Mini Assignment:** Switch MPM from prefork to worker and observe effect.
14. **Common Mistakes:** Forgetting to restart Apache after config changes; not securing loaded modules.
15. **Interview Questions:** *What are Apache MPMs and why choose one?*
16. **Quiz Questions:** *True/False on default Apache MPM on Ubuntu.*
17. **Lesson 17.3: SSL and Security in Apache** (Intermediate)
18. **Learning Outcome:** Configure SSL and basic security settings (e.g., `mod_security`).
19. **Topics Covered:** `SSLEngine on`, `SSLCertificateFile`, `.htaccess` basics.
20. **Hands-on Lab:** Enable `mod_security` or rate limiting (if available), and set up a self-signed cert.
21. **Mini Assignment:** Implement a simple password-protected directory using `.htpasswd`.
22. **Common Mistakes:** Insecure cipher suite; incorrect file permissions on `.htpasswd`.
23. **Interview Questions:** *How do you enable HTTPS in Apache?*
24. **Quiz Questions:** *MCQ on `AllowOverride` directive effect.*

Chapter 18: Artifact Repository Management

Learning Objective: Manage and store build artifacts using repository managers.
Skills Covered: Nexus Repository and JFrog Artifactory basics, hosting and retrieving artifacts.
Prerequisites: Understanding of build artifacts (JARs, NPM packages).
Estimated Time: 2–3 hours
Number of Lessons: 3

1. **Lesson 18.1: Sonatype Nexus Repository** (Intermediate)
2. **Learning Outcome:** Set up Nexus to host Maven and Docker artifacts.
3. **Topics Covered:** Nexus installation, creating hosted and proxy repositories, access control.
4. **Hands-on Lab:** Upload a sample Maven JAR to Nexus and configure `settings.xml` to pull it.
5. **Mini Assignment:** Configure a Docker registry in Nexus.
6. **Common Mistakes:** Publicly exposing repo without auth; not cleaning old snapshots.
7. **Interview Questions:** *What is a repository manager and why use one?*
8. **Quiz Questions:** *True/False on proxy vs hosted repos*.
9. **Lesson 18.2: JFrog Artifactory Basics** (Intermediate)
10. **Learning Outcome:** Use Artifactory for artifact storage (npm, PyPI, Docker).
11. **Topics Covered:** Repositories (local, remote, virtual), replication, API usage.
12. **Hands-on Lab:** Publish an NPM package to Artifactory and install it in a project.
13. **Mini Assignment:** Create a virtual repo aggregating two npm repos.
14. **Common Mistakes:** Mixing release and snapshot repos; not cleaning unused builds.
15. **Interview Questions:** *What is a virtual repository in Artifactory?*
16. **Quiz Questions:** *MCQ on Artifactory license tiers (OSS vs Pro)*.
17. **Lesson 18.3: Integrating Artifacts into CI/CD** (Intermediate)
18. **Learning Outcome:** Push and fetch artifacts as part of pipeline.
19. **Topics Covered:** Maven/Gradle config for Nexus/Artifactory, Docker push, automation.
20. **Hands-on Lab:** Configure Jenkins to publish a Docker image to Artifactory.
21. **Mini Assignment:** Set up npm publish to Nexus from CI.
22. **Common Mistakes:** Not handling credentials securely; forgetting to tag artifacts.
23. **Interview Questions:** *How does CI interact with an artifact repo?*
24. **Quiz Questions:** *True/False: CI should always pull external artifacts from central repos*.

Chapter 19: Build Tools (Maven & Gradle)

Learning Objective: Understand and use Java build tools within DevOps pipelines.
Skills Covered: Maven and Gradle fundamentals, project structure, dependency management, integration with CI/CD.
Prerequisites: Java project (sample) on hand.

Estimated Time: 2–3 hours
Number of Lessons: 4

1. **Lesson 19.1: Maven Basics** (Beginner)
2. **Learning Outcome:** Build Java projects using Maven (pom.xml, lifecycle)【33†L1-L4】.
3. **Topics Covered:** pom.xml structure, dependencies, `mvn clean install`, plugins.
4. **Hands-on Lab:** Create a simple Java project with Maven and add a dependency (e.g., JUnit).
5. **Mini Assignment:** Run unit tests with Maven and generate a JAR.
6. **Common Mistakes:** Wrong groupId/artifactId, missing `<scope>`.
7. **Interview Questions:** *What is the purpose of the POM file?*
8. **Quiz Questions:** *True/False on Maven phases (compile, test, package).*
9. **Lesson 19.2: Gradle Basics** (Beginner)
10. **Learning Outcome:** Build projects using Gradle (build.gradle, tasks).
11. **Topics Covered:** Gradle vs Maven, Groovy/Kotlin DSL, `./gradlew build`, dependency declaration.
12. **Hands-on Lab:** Convert the Maven project to Gradle (or start a new one), run tasks.
13. **Mini Assignment:** Add a new library and build the Gradle project.
14. **Common Mistakes:** Using old plugins; misunderstanding of `implementation` vs `compile`.
15. **Interview Questions:** *How do you add a dependency in Gradle?*
16. **Quiz Questions:** *MCQ on Gradle plugin vs task concept.*
17. **Lesson 19.3: Build Profiles and Environments** (Intermediate)
18. **Learning Outcome:** Use build profiles and parameters for different environments.
19. **Topics Covered:** Maven profiles, Gradle build variants, injecting environment variables.
20. **Hands-on Lab:** Define a Maven profile for a development vs production build (different DB URL).
21. **Mini Assignment:** Activate a Maven profile via command line.
22. **Common Mistakes:** Hardcoding environment info in code.
23. **Interview Questions:** *What is a Maven profile and when use it?*
24. **Quiz Questions:** *True/False on activating Gradle build types.*
25. **Lesson 19.4: Integrating Builds into CI/CD** (Intermediate)
26. **Learning Outcome:** Automate Maven/Gradle builds in Jenkins/GitHub Actions.

27. **Topics Covered:** Installing JDK on build agent, caching `.m2/repository` or Gradle cache, fail build on test errors.
28. **Hands-on Lab:** Configure a CI pipeline step to run `mvn package` or `gradle assemble`.
29. **Mini Assignment:** Speed up the build by using a cache of dependencies.
30. **Common Mistakes:** No caching (slow builds), not failing build when tests fail.
31. **Interview Questions:** *How do you fail a build on test failures?*
32. **Quiz Questions:** *MCQ on CI plugins (Maven/Gradle) available in Jenkins.*

Chapter 20: Testing in DevOps

Learning Objective: Cover automated testing practices within DevOps pipelines.
Skills Covered: Unit testing, integration testing, test automation tools, test coverage.
Prerequisites: Any programming language knowledge.
Estimated Time: 3–4 hours
Number of Lessons: 3

1. **Lesson 20.1: Unit Testing** (Intermediate)
2. **Learning Outcome:** Write and run unit tests as part of development.
3. **Topics Covered:** Testing frameworks (JUnit, PyTest, Jest, etc.), mock objects, test-driven development.
4. **Hands-on Lab:** Create unit tests for existing code and ensure all pass.
5. **Mini Assignment:** Increase code coverage by writing additional tests.
6. **Common Mistakes:** Tests dependent on external systems; flakey tests.
7. **Interview Questions:** *Why are unit tests important in CI/CD?*
8. **Quiz Questions:** *True/False on meaning of code coverage metric.*
9. **Lesson 20.2: Integration and End-to-End Testing** (Intermediate)
10. **Learning Outcome:** Automate integration tests (API, database).
11. **Topics Covered:** API testing (Postman, RestAssured), UI testing (Selenium), smoke tests.
12. **Hands-on Lab:** Write a test that calls the deployed service's API and verifies response.
13. **Mini Assignment:** Incorporate an integration test into the pipeline before deployment to prod.
14. **Common Mistakes:** Running slow integration tests on every commit (nightly is better).
15. **Interview Questions:** *What is an integration test?*
16. **Quiz Questions:** *MCQ on difference between unit and integration tests.*
17. **Lesson 20.3: Test Automation Tools** (Beginner)

18. **Learning Outcome:** Use automation tools and frameworks.
19. **Topics Covered:** CI integration (Jenkins, Actions), code coverage tools (JaCoCo, Istanbul), static analysis (SonarQube).
20. **Hands-on Lab:** Add code coverage report generation to CI and review it.
21. **Mini Assignment:** Analyze a SonarQube report and fix one identified issue.
22. **Common Mistakes:** Ignoring test reports, not prioritizing high-severity findings.
23. **Interview Questions:** *How would you measure test effectiveness?*
24. **Quiz Questions:** *True/False on static code analysis being part of testing.*

Chapter 21: Deployment Strategies

Learning Objective: Explore advanced deployment methods to minimize downtime and risk.
Skills Covered: Blue-Green, Canary, Rolling Updates, Rollbacks.
Prerequisites: Basics of Kubernetes or deployment platforms.
Estimated Time: 2–3 hours
Number of Lessons: 4

1. **Lesson 21.1: Blue-Green Deployment** (Intermediate)
2. **Learning Outcome:** Implement blue-green deployment for zero-downtime release.
3. **Topics Covered:** Duplication of environments (blue & green), traffic switching, database considerations.
4. **Hands-on Lab:** Deploy a new version to a 'green' environment, then shift load balancer from 'blue' to 'green'.
5. **Mini Assignment:** Practice rollback by switching traffic back to blue.
6. **Common Mistakes:** Data schema changes not backward-compatible.
7. **Interview Questions:** *How do you perform a blue-green deployment?*
8. **Quiz Questions:** *True/False: Blue-Green completely avoids downtime.*
9. **Lesson 21.2: Canary Deployment** (Intermediate)
10. **Learning Outcome:** Gradually release new version to subset of users.
11. **Topics Covered:** Traffic percentages, monitoring during canary, iterative rollouts.
12. **Hands-on Lab:** Use Kubernetes or a load balancer to route 10% traffic to new version (Canary).
13. **Mini Assignment:** Analyze logs/metrics to decide promotion or rollback.
14. **Common Mistakes:** Not monitoring canary sufficiently; exposing canary too broadly.
15. **Interview Questions:** *What is the advantage of Canary over Blue-Green?*
16. **Quiz Questions:** *Multiple choice on how to adjust traffic splits.*

17. **Lesson 21.3: Rolling Updates** (Intermediate)
18. **Learning Outcome:** Update servers incrementally (one by one).
19. **Topics Covered:** Kubernetes rolling update strategy, set max surge/unavailable, orchestration scripts.
20. **Hands-on Lab:** Perform a rolling update in a Deployment with `kubectl rollout`.
21. **Mini Assignment:** Configure Kubernetes Deployment strategy to allow 2 new pods and 1 unavailable.
22. **Common Mistakes:** Not having readiness probes (causing traffic to non-ready pods).
23. **Interview Questions:** *When use rolling updates?*
24. **Quiz Questions:** *True/False on `maxSurge` vs `maxUnavailable`.*
25. **Lesson 21.4: Automated Rollback Strategies** (Intermediate)
26. **Learning Outcome:** Automate rollback on failure (or manual triggers).
27. **Topics Covered:** CrashLoopBackOff handling, `kubectl rollout undo`, CI/CD pipeline rollback stage.
28. **Hands-on Lab:** Simulate a bad release and rollback in Kubernetes or Jenkins.
29. **Mini Assignment:** Integrate a health check that triggers rollback if threshold breaches.
30. **Common Mistakes:** Rolling back without fixing root cause; not notifying teams on rollback.
31. **Interview Questions:** *How do you automatically rollback a failed deployment?*
32. **Quiz Questions:** *MCQ on `kubectl rollout undo` usage.*

Chapter 22: High Availability and Disaster Recovery

Learning Objective: Ensure applications remain available and recoverable.
Skills Covered: Auto Scaling, Load Balancers, backups, failover, disaster recovery planning.
Prerequisites: Cloud basics (for scaling).
Estimated Time: 3–4 hours
Number of Lessons: 4

1. **Lesson 22.1: Auto Scaling** (Intermediate)
2. **Learning Outcome:** Configure auto scaling of resources (EC2/AWS Auto Scaling Groups, Kubernetes HPA).
3. **Topics Covered:** Auto Scaling Groups, launch configs, scaling policies (CPU, schedule); K8s Horizontal Pod Autoscaler.

4. **Hands-on Lab:** Set up an AWS Auto Scaling Group that spins instances on CPU > 70%.
5. **Mini Assignment:** Trigger scale up/down manually (e.g., increase load).
6. **Common Mistakes:** Too aggressive scaling causing churn; not scaling down (cost).
7. **Interview Questions:** *What metrics can trigger auto scaling?*
8. **Quiz Questions:** *True/False on difference between AWS ASG and ELB.*

9. **Lesson 22.2: Multi-AZ and Load Balancing** (Intermediate)

10. **Learning Outcome:** Distribute applications across multiple AZs for availability.

11. **Topics Covered:** Cross-AZ DB clusters, multi-AZ EC2, ELB health checks, active/active vs active/passive.
12. **Hands-on Lab:** Configure an ELB across two AZs, verify failover when one AZ is disabled (simulate via instance shutdown).
13. **Mini Assignment:** Implement an RDS Multi-AZ setup.
14. **Common Mistakes:** Not duplicating stateful data across AZs.
15. **Interview Questions:** *Why multi-AZ deployment is recommended?*
16. **Quiz Questions:** *MCQ on ALB vs NLB for high availability.*

17. **Lesson 22.3: Disaster Recovery Strategies** (Intermediate)

18. **Learning Outcome:** Plan for catastrophic failures and data loss.

19. **Topics Covered:** Backup types (full, incremental), RTO (Recovery Time Objective), RPO (Recovery Point Objective).
20. **Hands-on Lab:** Set up automated backups (EC2 snapshots, RDS snapshots) and test restore.
21. **Mini Assignment:** Create an AMI before patching and restore from it after failure.
22. **Common Mistakes:** No off-site backups; no DR testing.
23. **Interview Questions:** *Define RPO vs RTO.*
24. **Quiz Questions:** *True/False: Backups alone ensure DR.*

25. **Lesson 22.4: Data Replication and Backup** (Intermediate)

26. **Learning Outcome:** Configure continuous data replication for minimal downtime.

27. **Topics Covered:** Database replication (MySQL master-slave), S3 replication, cross-region replication.
28. **Hands-on Lab:** Enable S3 bucket cross-region replication or Aurora read replica.
29. **Mini Assignment:** Script an incremental backup (e.g., using `rsync`) to a secondary server.
30. **Common Mistakes:** Replication lag, forgetting to failover DNS.
31. **Interview Questions:** *How do read-replicas help in HA?*

32. **Quiz Questions:** *MCQ on differences between backup and replication*.

Chapter 23: Performance Optimization

Learning Objective: Optimize system and application performance.
Skills Covered: Caching strategies, CDN, database indexing, profiling, tuning.
Prerequisites: Basic understanding of the application stack.
Estimated Time: 2–3 hours
Number of Lessons: 3

1. **Lesson 23.1: Caching and Content Delivery** (Intermediate)
2. **Learning Outcome:** Use caching layers and CDNs to reduce load and latency.
3. **Topics Covered:** In-memory caches (Redis, Memcached), HTTP caching headers, AWS CloudFront or CDN integration.
4. **Hands-on Lab:** Configure Redis as a cache for a web application or set cache headers on static assets.
5. **Mini Assignment:** Set up CloudFront for an S3 bucket website.
6. **Common Mistakes:** Not invalidating cache on deploy; cache coherence issues.
7. **Interview Questions:** *When would you use a CDN?*
8. **Quiz Questions:** *True/False on cache eviction policies*.
9. **Lesson 23.2: Database and Query Optimization** (Intermediate)
10. **Learning Outcome:** Tune database performance with indexing and query review.
11. **Topics Covered:** Using EXPLAIN plan, adding indexes, connection pooling.
12. **Hands-on Lab:** Add an index to a slow query and measure performance improvement.
13. **Mini Assignment:** Identify a slow API call and optimize its database query.
14. **Common Mistakes:** Over-indexing (slow writes), not normalizing data.
15. **Interview Questions:** *What are database indexing trade-offs?*
16. **Quiz Questions:** *MCQ on ACID vs BASE in databases*.
17. **Lesson 23.3: Monitoring and Profiling Performance** (Intermediate)
18. **Learning Outcome:** Profile CPU/memory and identify bottlenecks.
19. **Topics Covered:** `top`/`perf`/`strace` (Linux profiling), APM tools (New Relic, Jaeger for tracing).
20. **Hands-on Lab:** Profile a CPU-heavy application using `perf` or an APM, identify hotspot.
21. **Mini Assignment:** Implement trace logging to pinpoint latency in code.
22. **Common Mistakes:** Not profiling before optimizing; optimizing wrong code path.

23. **Interview Questions:** *How do you find memory leaks?*
24. **Quiz Questions:** *True/False on correlation between profiling and performance improvement.*

Chapter 24: Troubleshooting and Incident Response

Learning Objective: Develop skills to quickly diagnose and resolve system issues.
Skills Covered: Troubleshooting methodology, log analysis, incident management, root cause analysis.
Prerequisites: Access to logs/monitoring from previous chapters.
Estimated Time: 3–4 hours
Number of Lessons: 3

1. **Lesson 24.1: System and Application Troubleshooting** (Intermediate)
2. **Learning Outcome:** Use logs, metrics, and commands to diagnose issues.
3. **Topics Covered:** Log analysis (grep, Kibana), heap dumps, analyzing crashes, journalctl.
4. **Hands-on Lab:** Debug a service failure by examining its logs and resource usage.
5. **Mini Assignment:** Document the steps taken and solution found for a simulated outage.
6. **Common Mistakes:** Restarting without understanding cause; blaming random components.
7. **Interview Questions:** *What's your approach to diagnosing a slow web app?*
8. **Quiz Questions:** *MCQ on tools (strace vs dmesg).*
9. **Lesson 24.2: CI/CD Pipeline Debugging** (Intermediate)
10. **Learning Outcome:** Identify and fix failures in automated pipelines.
11. **Topics Covered:** Reading Jenkins/GitHub Actions logs, artifact debugging, environment differences (DEV vs PROD).
12. **Hands-on Lab:** Induce a failure in a pipeline (e.g., missing env var) and fix it.
13. **Mini Assignment:** Check out a failed build log and explain the error.
14. **Common Mistakes:** Making pipeline changes without testing locally, ignoring flaky tests.
15. **Interview Questions:** *How do you handle a pipeline that fails intermittently?*
16. **Quiz Questions:** *True/False: CI log retention is irrelevant for debugging.*
17. **Lesson 24.3: Production Incident Management** (Intermediate)
18. **Learning Outcome:** Follow best practices for incident response (alerting, runbooks).

19. **Topics Covered:** Incident triage (severity, communication), postmortems, blameless culture.
20. **Hands-on Lab:** Simulate an incident (e.g., service down) and write a brief postmortem outline.
21. **Mini Assignment:** Create a checklist of steps to follow when an alert fires.
22. **Common Mistakes:** Panic without plan, skipping documentation.
23. **Interview Questions:** *What is a postmortem and why do it?*
24. **Quiz Questions:** *MCQ on key components of an incident review.*

Chapter 25: DevOps Best Practices and Culture

Learning Objective: Reinforce best practices and soft skills in DevOps.
Skills Covered: Collaboration, documentation, continuous learning, code review, agile ceremonies.
Prerequisites: Completed technical chapters above.
Estimated Time: 2–3 hours
Number of Lessons: 3

1. **Lesson 25.1: Collaboration and Communication** (Beginner)
2. **Learning Outcome:** Emphasize cross-team collaboration and open communication.
3. **Topics Covered:** Agile ceremonies (daily standups, retrospectives), shared ownership, DevOps culture.
4. **Hands-on Lab:** Practice a retrospective meeting in your team on a completed sprint.
5. **Mini Assignment:** Write a section of team documentation (e.g., on wiki).
6. **Common Mistakes:** Siloing information in individuals, skipping feedback loops.
7. **Interview Questions:** *How do you handle conflicts in a team?*
8. **Quiz Questions:** *True/False on pair programming and collaboration.*
9. **Lesson 25.2: Documentation and Version Control** (Beginner)
10. **Learning Outcome:** Maintain up-to-date documentation and version everything.
11. **Topics Covered:** README importance, architecture diagrams, keeping infra code in Git.
12. **Hands-on Lab:** Update project README with setup instructions.
13. **Mini Assignment:** Draft a document describing your CI/CD pipeline.
14. **Common Mistakes:** Outdated docs; infra changes not in code repo.
15. **Interview Questions:** *Why is documentation critical in DevOps?*
16. **Quiz Questions:** *MCQ on different types of documentation (design, runbooks).*
17. **Lesson 25.3: Continuous Learning and Improvement** (Beginner)

18. **Learning Outcome:** Cultivate a culture of learning (blameless, postmortems, innovation time).
19. **Topics Covered:** Metrics review (e.g., DORA), encouraging skill development, tech talks.
20. **Hands-on Lab:** (Activity) Identify one new DevOps tool to learn and set a mini-project.
21. **Mini Assignment:** Present a short tech talk on a DevOps topic.
22. **Common Mistakes:** Assuming you know it all; not addressing failures.
23. **Interview Questions:** *How do you stay updated with new DevOps trends?*
24. **Quiz Questions:** *True/False on importance of postmortems even when things go right.*

Chapter 26: Site Reliability Engineering (SRE) Fundamentals

Learning Objective: Introduce SRE concepts as related to DevOps and reliability.
Skills Covered: SLO/SLI/SLA, error budgets, toil reduction, monitoring best practices.
Prerequisites: Understanding of monitoring (Chapter 13) and operations.
Estimated Time: 3–4 hours
Number of Lessons: 4

1. **Lesson 26.1: Introduction to SRE** (Beginner)
2. **Learning Outcome:** Define SRE and its philosophy (ops as code)【113†L100-L104】.
3. **Topics Covered:** SRE vs DevOps, reliability goals, SRE teams, Google's definition.
4. **Hands-on Lab:** (Discussion) Identify tasks in current workflow that could be automated (toil).
5. **Mini Assignment:** Calculate hypothetical SLI for an uptime metric.
6. **Common Mistakes:** Mistaking SRE for only monitoring or only incident response.
7. **Interview Questions:** *What is the main goal of SRE?*
8. **Quiz Questions:** *True/False: SRE says to eliminate all manual work.*
9. **Lesson 26.2: SLOs, SLIs, and SLAs** (Intermediate)
10. **Learning Outcome:** Set reliability targets and measure service health.
11. **Topics Covered:** Service Level Indicators (metrics like error rate), Objectives (target values), Agreements (customer commitments).
12. **Hands-on Lab:** Define an SLO (e.g., 99.9% success) and simulate tracking it with Prometheus.
13. **Mini Assignment:** Draft an SLO for API response time.
14. **Common Mistakes:** Setting SLOs too high without justification; ignoring error budgets.

15. **Interview Questions:** *Explain difference between SLA and SLO.*
16. **Quiz Questions:** *Multiple choice on what constitutes an SLI.*
17. **Lesson 26.3: Error Budgets and Release Policies** (Intermediate)
18. **Learning Outcome:** Use error budgets to balance release velocity and reliability.
19. **Topics Covered:** Calculating error budget, halting releases when budget spent, engineering work for reliability.
20. **Hands-on Lab:** Simulate budget consumption (e.g., 1% downtime) and decide on pausing a release.
21. **Mini Assignment:** Discuss how your team could use error budgets.
22. **Common Mistakes:** Viewing availability as all-or-nothing; ignoring small outages.
23. **Interview Questions:** *What do you do when you exceed your error budget?*
24. **Quiz Questions:** *True/False: Error budget can be negative.*
25. **Lesson 26.4: Reducing Toil and Automating** (Intermediate)
26. **Learning Outcome:** Identify repetitive tasks (toil) and reduce them via automation.
27. **Topics Covered:** Toil vs project work, runbooks automation, monitoring improvements.
28. **Hands-on Lab:** Write a script or Ansible playbook to automate a routine admin task.
29. **Mini Assignment:** Measure time spent on a manual task and set a target to automate it.
30. **Common Mistakes:** Automating one-off tasks; neglecting documentation.
31. **Interview Questions:** *What is 'toil' in SRE terms?*
32. **Quiz Questions:** *MCQ on examples of toil vs project work.*

Chapter 27: Production Deployment and Release Management

Learning Objective: Plan and execute the final release of applications into production safely.
Skills Covered: Production readiness review, deployment pipelines, observability during rollout, chaos testing introduction.
Prerequisites: Prior CI/CD and deployment strategy knowledge.
Estimated Time: 2–3 hours
Number of Lessons: 3

1. **Lesson 27.1: Production Readiness Checklist** (Intermediate)
2. **Learning Outcome:** Ensure all pre-deployment criteria are met (monitoring, logging, backups).
3. **Topics Covered:** Readiness (health checks, capacity planning), checklist items for infra and app.

4. **Hands-on Lab:** Review your application against a provided PRD or checklist.
5. **Mini Assignment:** Fill out a template production readiness form for your app.
6. **Common Mistakes:** Skipping readiness review; deploying without health checks.
7. **Interview Questions:** *What is included in a production readiness review?*
8. **Quiz Questions:** *True/False on importance of canary testing in prod.*

9. **Lesson 27.2: Release Orchestration** (Intermediate)

10. **Learning Outcome:** Coordinate steps for a release: CI build, staging test, approval, production deploy.

11. **Topics Covered:** Release flow in Jenkins/GitHub, version tagging, rollback plans.
12. **Hands-on Lab:** Tag a release in Git, trigger the pipeline to deploy to staging then production.
13. **Mini Assignment:** Document steps for a hotfix deployment vs regular release.
14. **Common Mistakes:** Lack of rollback strategy; ad-hoc scripts outside pipeline.
15. **Interview Questions:** *How do you handle a critical hotfix in production?*
16. **Quiz Questions:** *MCQ on semantic versioning importance.*

17. **Lesson 27.3: Chaos Engineering Introduction** (Beginner)

18. **Learning Outcome:** Understand basics of intentionally disrupting systems to improve resilience.

19. **Topics Covered:** Chaos tools (Chaos Monkey), running game days, experimentation culture.
20. **Hands-on Lab:** (Optional) Use a tool like `docker kill` randomly in staging to simulate failure.
21. **Mini Assignment:** Plan a simple chaos experiment (e.g., shutdown one node).
22. **Common Mistakes:** Causing major outages without safety net; doing it in production without prep.
23. **Interview Questions:** *What is the purpose of chaos engineering?*
24. **Quiz Questions:** *True/False: Chaos engineering eliminates all failures.*

Chapter 28: Resume and Career Preparation

Learning Objective: Equip learners to present DevOps skills effectively for job search.
Skills Covered: Crafting DevOps-focused resume, highlighting projects, interview readiness.
Prerequisites: Completed course (for content).
Estimated Time: 1–2 hours
Number of Lessons: 2

1. **Lesson 28.1: Building a DevOps Resume** (Beginner)

2. **Learning Outcome:** Create a resume emphasizing DevOps tools and projects.
3. **Topics Covered:** Resume structure, technical vs soft skills, keyword optimization (CI/CD, cloud, etc.).
4. **Hands-on Lab:** Draft or revise your resume based on learned topics.
5. **Mini Assignment:** Peer-review a DevOps resume for improvements.
6. **Common Mistakes:** Generic descriptions, not quantifying impact.
7. **Interview Questions:** *What DevOps tools are on your resume?*
8. **Quiz Questions:** *Checklist: include Git, Docker, K8s on resume.*
9. **Lesson 28.2: Interviewing for DevOps Roles** (Beginner)
10. **Learning Outcome:** Prepare for technical and behavioral DevOps interviews.
11. **Topics Covered:** Common questions (technical and situational), demonstrating projects, communication tips.
12. **Hands-on Lab:** Conduct a mock interview with a peer using sample DevOps questions.
13. **Mini Assignment:** Prepare answers to 5 common interview questions.
14. **Common Mistakes:** Rambling answers, lacking examples, technical jargon without clarity.
15. **Interview Questions:** *Write an elevator pitch for yourself as a DevOps engineer.*
16. **Quiz Questions:** *MCQ on STAR method for behavioral answers.*

Chapter 29: DevOps Interview Preparation

Learning Objective: Provide practice for common DevOps interview scenarios.
Skills Covered: Answering technical questions, scenario-based problem-solving, whiteboarding.
Prerequisites: Course knowledge.
Estimated Time: 2–3 hours
Number of Lessons: 3

1. **Lesson 29.1: DevOps Technical Questions** (Intermediate)
2. **Learning Outcome:** Handle questions on CI/CD tools, Linux, scripting, and networking.
3. **Topics Covered:** Sample questions (e.g., “How to resolve merge conflict?”, “Explain pipeline stages”), whiteboard answers.
4. **Hands-on Lab:** Practice explaining a technical concept (e.g., Kubernetes Service) to a peer.
5. **Mini Assignment:** Write out answers to 5 random technical questions.
6. **Common Mistakes:** Giving incomplete answers, forgetting to state assumptions.
7. **Interview Questions:** *(Presented question examples as interview prep.)*

8. **Quiz Questions:** *Mock Q&A: choose best answer from multiple choices*.
9. **Lesson 29.2: Scenario-Based Questions** (Intermediate)
10. **Learning Outcome:** Solve DevOps problems given real-world scenarios (incident response, scaling, optimization).
11. **Topics Covered:** "What if" questions (e.g., server down, CI failing), step-by-step reasoning.
12. **Hands-on Lab:** Role-play: assign a role-play scenario and discuss solution approach.
13. **Mini Assignment:** Brainstorm answers for 3 scenario prompts.
14. **Common Mistakes:** Overlooking simple fixes, not verifying understanding of scenario.
15. **Interview Questions:** *(These are practice scenarios.)*
16. **Quiz Questions:** *Multiple choice on best next steps in given scenario*.
17. **Lesson 29.3: System Design and Architecture Questions** (Intermediate)
18. **Learning Outcome:** Design high-level architecture diagrams for scalable systems.
19. **Topics Covered:** Designing a fault-tolerant web app on AWS/Kubernetes, whiteboarding tools.
20. **Hands-on Lab:** Sketch a design for a highly available service with load balancing and databases.
21. **Mini Assignment:** Prepare a sample architecture diagram for a given requirement.
22. **Common Mistakes:** Overcomplicating, ignoring cost or maintenance.
23. **Interview Questions:** *Design a CI/CD system for a microservices app*.
24. **Quiz Questions:** *MCQ on AWS architecture components*.

Chapter 30: Final Capstone Projects

Learning Objective: Apply all learned skills in real-world project deployments.
Skills Covered: End-to-end DevOps implementation.
Prerequisites: Completed course (capstone draws on all chapters).
Estimated Time: Varies per project (20+ hours total)
Number of Projects: 10 major projects (each can be divided into lessons if needed)

Capstone Projects:

- **Linux Server Setup:** Configure a Linux server with SSH, users, services, and security.
- **Git Workflow Project:** Implement Git branching, merging, and pull requests in a team exercise.

- **Dockerize a React Application:** Containerize a front-end app with Dockerfile and serve via Nginx.
- **Dockerize a Node.js API:** Containerize a Node API with MongoDB and demonstrate `docker-compose` setup.
- **Docker Compose Full-Stack:** Build a multi-container setup (React + Node + DB) using Docker Compose.
- **Jenkins CI/CD Pipeline:** Create a Jenkins pipeline to build, test, and deploy a sample app to staging.
- **GitHub Actions CI/CD:** Automate build/test/deploy workflow on GitHub for a repository.
- **Kubernetes Deployment:** Deploy the full-stack app to a Kubernetes cluster with Helm.
- **AWS Infrastructure with Terraform:** Provision the above app environment on AWS using Terraform (EC2, RDS, S3).
- **Ansible Server Automation:** Automate the configuration of a set of servers using Ansible playbooks/roles.
- **Monitoring and Logging Implementation:** Set up Prometheus/Grafana monitoring and ELK logging for the deployed apps.
- **Complete MERN Stack Production Deployment:** End-to-end deploy a MERN (MongoDB-Express-React-Node) stack with CI/CD, Docker, Kubernetes, and monitoring.
  (Projects can be scaled in complexity and split into sub-tasks or lessons.)

Additional Resources and Roadmaps

- **DevOps Roadmap:** A comprehensive learning path covering all key DevOps domains (version control, CI/CD, cloud, containers, etc.).
- **Technology Roadmaps:** Detailed roadmaps for each major technology: Linux, Docker, Kubernetes, AWS, Terraform, CI/CD pipelines, security.
- **Career Path:** Guidance on roles (DevOps Engineer, SRE, Platform Engineer) and skill progression in a DevOps career.
- **Best Practices Checklist:** A DevOps checklist including infrastructure as code, code reviews, automation, security scans, and continuous monitoring.
- **Practice Questions:** 300+ practice questions covering technical concepts and problem-solving.
- **Interview Questions:** 150+ curated DevOps interview questions (technical and behavioral).
- **Hands-on Labs:** 50+ lab exercises reinforcing chapter topics.
- **Major Projects:** 20+ comprehensive projects (outlined above) for real-world experience.

- **Production Deployment Checklist:** Steps and items to verify before and after production releases.
- **Troubleshooting Guide:** Common troubleshooting scenarios and resolution steps.
- **Common Interview Scenarios:** Typical interview case studies and role-play scenarios.
- **Recommended Books:** E.g., *The Phoenix Project*, *The DevOps Handbook*, *Site Reliability Engineering*, *Kubernetes Up & Running*.
- **Official Documentation:** Links to official docs for all tools covered (Linux, Docker, Kubernetes, AWS, Terraform, Ansible, etc.).
- **GitHub Repositories:** Sample open-source DevOps projects and configurations (e.g., GitOps demos, Terraform AWS examples).
- **Open Source Tools:** List of relevant OSS projects (Prometheus, Grafana, Jenkins, ArgoCD, etc.).
- **Learning Resources:** Official tutorials, community blogs, and courses (e.g., Kubernetes docs, AWS workshops, Docker getting started).
